

基于改进牛顿法的舞龙队轨迹问题

摘要

探究“板凳龙”盘龙技术之美，为中华优秀传统文化的继承与发扬注入生机与活力。本文围绕板凳龙盘龙时的运动学过程，建立起把手位置模型、舞龙队速度模型，基于板凳运动中的刚体限制、板凳之间碰撞情况等约束，利用**速度分解、分离轴定理、改进牛顿法、粗细网格遍历**对不同优化目标下舞龙队的位置和速度进行了求解。

针对问题一，对于整个舞龙队位置的求解，首先根据龙头的行进速度，通过**阿基米德螺线弧长公式**得到积分的**原函数**，从而反解每个时刻龙头的位置见正文所示；再通过舞龙队间连接板凳长度建立距离约束，使用基于割线近似切线、强制首步控制、小步长搜索、负向重搜机制的**改进牛顿法**进行迭代，对于整个舞龙队速度的求解，通过**速度沿着板凳方向分解速度相同**的性质决定了前后板凳龙的速度关系。最后对龙头初速度进行灵敏度分析，得到在同一舞龙队位置下龙头把手速度与其他各点速度**比值为定值**。

问题二在问题一的基础上，以最大化终止时刻为优化目标，根据各把手中心坐标位置求出各板凳龙四周顶点坐标，建立基于**分离轴定理**的板凳碰撞判断模型，在板凳之间不发生碰撞的约束下，使用粗细网格遍历求解了舞龙队盘入的终止时刻为**412.474s**，并给出了此时舞龙队的位置和速度。

针对问题三，根据前文分析，舞龙队越靠近螺线中心，板凳之间越容易发生碰撞。在满足上述约束的前提下，以最小化螺距为优化目标，对龙头前把手刚进入调头空间时利用基于分离轴定理的板凳碰撞判断模型进行碰撞判断，使用粗细网格遍历从而确定了最小螺距为**41.713cm**。

针对问题四，本文首先确定切入点与切出点的极角相等，基于此极角构建决策变量，以调头空间范围、盘出螺线与盘入曲线中心对称、调头曲线各部分相切、板凳运动中的**刚体约束**、板凳之间碰撞情况为约束条件，优化最短调头曲线，根据反解距离方程对把手位置进行分类，然后求解得到在极角大于**6.3rad**时不再会发生碰撞，在此基础上求解得到极角为**15.32rad**，调头曲线长度为**12.51m**，相对于调整前优化了**8.15%**。

问题五在问题四的基础上，舞龙队以问题四中的路径行进，本文以龙头的行进速度为决策变量，以舞龙队行进过程中各把手速度约束建立最大龙头行进速度模型进行求解，根据问题一中灵敏度分析结论，直接根据比值关系求解得到龙头把手速度为**0.45m/s**，同时代入原模型进行验证。

本文利用数学方法求解积分原函数，避免了反解积分函数的问题。通过对牛顿法进行改进，把有约束优化问题转化为**无约束方程问题**。利用分离轴定理，准确且快速地判断图形碰撞关系。使用粗细网络遍历，提高精度的同时减少了计算量。对模型引入刚体约束与碰撞约束，符合实际情况。

关键词：速度分解 分离轴定理 改进牛顿法 粗细网格遍历

一、问题重述

1.1 问题背景

“板凳龙”是一项起源于浙闽地区的民俗活动。舞龙队由数个板凳首尾相连组成，龙头领队，龙身与龙尾相继盘旋形成圆盘状。

本问中的板凳龙由 223 节板凳组成。龙头的板凳长 341cm，龙身和龙尾的板凳长 220cm，龙头、龙身和龙尾的板凳板宽均为 30cm。把手连接相邻的两个板凳，孔位于板宽的中心，孔的中心距离最近的板头 27.5cm，孔径 5.5cm。

1.2 问题提出

为探究蕴含在“板凳龙”中的数学知识，根据题中所给信息建立模型解决以下问题：

问题一 舞龙队沿螺线顺时针盘入，已知螺距且所有把手的中心都位于螺线上。根据龙头初始位置和运动速度，计算从 0 到 300 秒期间，每秒舞龙队的位置和速度。并且求解某些时刻时，指定节板凳龙的位置和速度。

问题二 舞龙队按照问题 1 设定的螺线盘入，即将发生碰撞的时刻作为舞龙队盘入的终止时刻，计算此时舞龙队的位置和速度，求解此时指定节板凳龙的位置和速度。

问题三 规定一圆形区域为调头区，以螺线中心为圆心，直径为 9 m。计算舞龙行进的最小螺距，使龙头的前把手能够沿着相应的螺线盘入该区域的边界。

问题四 已知龙头行进速度始终 1 m/s。舞龙队按问题 3 调头，路径由两段圆弧相切连接，盘入圆弧的半径是盘出圆弧半径的 2 倍。在保持相切的条件下调整圆弧，使得调头曲线变短。计算 -100s 到 100s 期间，每秒舞龙队的位置和速度。并且求解某些时刻，指定节板凳龙的位置和速度。

问题五 舞龙队按照问题 4 得到的路径运动，龙头运行速度保持不变，请确定龙头的最大运行速度，使得舞龙队各把手的速度均不超过 2 m/s。

二、问题分析

2.1 针对第一个问题

问题一要求在舞龙队沿等距螺线顺时针盘入、龙头前把手保持 1m/s 行进速度的情况下，求解从初始时刻到 300s 过程中每秒整个舞龙队的位置和速度。为描述舞龙队行进情况，本文分别建立了极坐标系与直角坐标系。对于整个舞龙队位置的求解，可分为两部分进行求解：第一部分为根据龙头的行进速度，通过阿基米德螺线弧长公式反解每个时刻龙头的位置；第二部分为根据龙头、龙身、龙尾间连接板凳长度建立距离约束，迭代求解 t 时刻下整个舞龙队的位置。对于整个舞龙队速度的求解，通过该盘入螺线的切向量得到速度分量间夹角关系，从而根据同一时刻前后板凳龙的速度关系对该时刻整个舞龙对的速度进行迭代求解。

2.2 针对第二个问题

本问要求舞龙队以问题一设定的螺线盘入轨迹行进，在板凳之间不发生碰撞的约束下，求解舞龙队盘入的终止时刻，并给出此时舞龙队的位置和速度。首先需根据各把手中心坐标位置求出各板凳龙四周顶点坐标；然后建立基于分离轴定理的板凳碰撞判断模型进行求解。

2.3 针对第三个问题

本文要求在龙头前把手能够沿着螺线盘入到调头空间的条件下,求解舞龙队盘入螺线的最小螺距。根据问题二分析可得,舞龙队越靠近螺线中心,板凳之间越容易发生碰撞。为使得龙头前把手顺利盘入调头空间,板凳之间不发生碰撞,需对不同螺距下龙头前把手进入调头空间时各板凳间碰撞情况进行判断,从而确定最小螺距。

2.4 针对第四个问题

本题的核心为目标优化问题,在求解在满足题中所给条件的圆弧后,通过调整圆弧,实现调头曲线尽可能短的目标。本文以切入点与切出点的极角为决策变量,以调头空间范围、盘出螺线与盘入曲线中心对称、各部分相切、板凳运动中的刚体约束、板凳之间碰撞情况为约束条件,最终建立调头曲线优化模型给出圆弧的调整方案。

2.5 针对第五个问题

本文要求在舞龙队沿问题四中路径行进,龙头行进速度保持不变的情况下,确定满足舞龙队个把手的速度均不超过 2m/s 约束的最大龙头行进速度。为解决上述问题,本文以龙头的行进速度为决策变量,以舞龙队行进过程中各把手速度约束建立最大龙头行进速度模型进行求解。

三、模型假设

为保证模型的合理性和准确性,本文结合实际情况,排除部分干扰因素,提出以下假设:

1. 假设板凳龙的板凳均为质量均匀分布、重心位于几何中心的刚体;
2. 假设板凳龙在二维平面上运动,将板凳视为二维平面的几何体;
3. 假设忽略如空气阻力、重力、板凳间摩擦力等对运动过程中的影响;
4. 假设龙头的速度始终为 1m/s , 不受干扰;
5. 假设板凳龙的把手中心的运动路径与给定螺线方程完全一致。

四、符号说明

序号	符号	符号说明及意义
1	θ	极角
2	L	弧线长度
3	d_0	龙头把手间的距离
4	d_n	龙身、龙尾把手间的距离
5	α_n	第 n 、 $n-1$ 节把手中心连线与螺线切线夹角
6	d_{o1o2}	调头圆弧两圆心间的距离
7	v_n	第 n 节龙身的行进速度

五、问题一模型的建立与求解

问题一要求在舞龙队沿已知的等距螺线顺时针盘入、龙头前把手保持 1m/s 行进速度的情况下，求解从初始时刻到 300s 过程中，每秒整个舞龙队的位置和速度。等距螺线是一个点匀速离开一个固定点的同时又以固定角速度绕该固定点转动而产生的轨迹，即阿基米德螺线^[1]。本文先建立把手位置模型得到 t 时刻下整个舞龙队的位置，再通过速度分解得到当前时刻舞龙队的速度。为简化模型，本文将舞龙队盘入的运动视作在二维平面上的运动。

本问中舞龙队沿等距螺线的轨迹运动。为描述舞龙队行进情况，在舞龙的运动平面上，建立以螺线中心为极点，以极点向右的水平直线为极轴的极坐标系。从极点向平面内任意一点的连线距离记为极径 r ，由极轴和从极点向该点连线间的夹角记为极角 θ 。由此，舞龙队沿螺距为 b 的等距螺线的运动轨迹在极坐标系下表达为：

$$r = b\theta \quad (1)$$

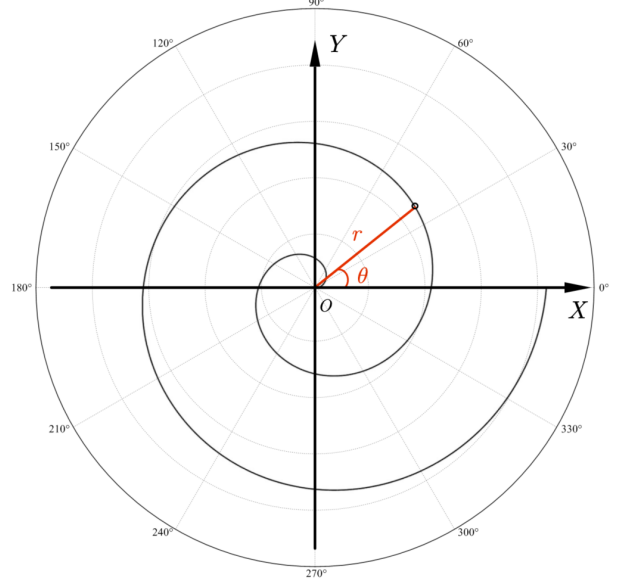


图 1 阿基米德螺旋在不同坐标系下

5.1 极坐标系与直角坐标系间转换

将极坐标系与直角坐标系进行转换。以螺线中点为原点，水平方向向右为 x 轴正方向，竖直方向向上为 y 轴正方向，建立平面直角坐标系。极坐标 (r, θ) 和直角坐标 (x, y) 的转换关系如下：

$$\begin{cases} r = \sqrt{x^2 + y^2} \\ \tan \theta = \frac{y}{x} (x \neq 0) \end{cases} \quad (2)$$

反之，将直角坐标系 (x, y) 转换为极坐标 (r, θ) 则有：

$$\begin{cases} x = b\theta \cos \theta \\ y = b\theta \sin \theta \end{cases} \quad (3)$$

可得该等距螺线在直角坐标系下的表达式为：

$$\begin{cases} x = b\theta \cos \theta \\ y = b\theta \sin \theta \end{cases} \quad (4)$$

5.2 把手位置模型的建立

舞龙队沿等距螺线顺时针盘入，各把手位置始终位于螺线上，通过龙头位置即可确定整个舞龙队的位置。根据题意分析可得，在舞龙队运动过程中整个舞龙队的位置可分为两部分进行求解：第一部分为根据龙头的行进速度，通过阿基米德螺线弧长公式反解每个时刻龙头的位置；第二部分为根据龙头、龙身、龙尾间连接板凳长度建立距离约束，迭代求解 t 时刻下整个舞龙队的位置。由此可以得到从初始时刻到 300s 过程中每秒整个舞龙队的位置。

5.2.1 基于弧长公式的坐标反解模型

为通过弧长公式反解 t 时刻龙头的极坐标，对阿基米德螺线弧长公式的推导如下：
极坐标下曲线的弧长公式为：

$$L = \int_{\theta_2}^{\theta_1} \sqrt{r^2 + \left(\frac{dr}{d\theta}\right)^2} d\theta \quad (5)$$

已知阿基米德螺线在极坐标系中的表达式为： $r = b\theta$
其两边对 θ 求导可得：

$$\frac{dr}{d\theta} = b \quad (6)$$

将其代入阿基米德螺线极坐标公式中可得阿基米德螺线长度在极坐标中表达为：

$$L = \int_{\theta_2}^{\theta_1} \sqrt{(b\theta)^2 + b^2} d\theta = b \int_{\theta_2}^{\theta_1} \sqrt{\theta^2 + 1} d\theta \quad (7)$$

其中， θ_1 为螺线积分起点的极角，即 A 点处的极角； θ_2 为 t 时刻龙头处极角。
利用三角代换对上式化简，设 $\theta = \tan u$ ，有 $d\theta = \sec^2 u du$ ，则：

$$\sqrt{\theta^2 + 1} = \sqrt{\tan^2 u + 1} = \sec u \quad (8)$$

将 $\sqrt{\theta^2 + 1} = \sec u$ 代入螺线长度公式，该积分化为：

$$L = b \int \sec u \sec^2 u du = b \int \sec^3 u du \quad (9)$$

对 $\int \sec^3 u du$ 进行分部积分可得：

$$\int \sec^3 u du = \frac{1}{2} \sec u \tan u + \frac{1}{2} \ln |\sec u + \tan u| + c \quad (10)$$

由 $\tan u = \theta$, $\sec u = \sqrt{1 + \theta^2}$ ，代入可得：

$$\int \sec^3 u du = \frac{1}{2} \theta \sqrt{\theta^2 + 1} + \frac{1}{2} \ln(\theta + \sqrt{\theta^2 + 1}) + c \quad (11)$$

综上所述，阿基米德螺线弧长公式为：

$$L = b \left[\frac{1}{2} \theta \sqrt{\theta^2 + 1} + \frac{1}{2} \ln(\theta + \sqrt{\theta^2 + 1}) \right]_{\theta_2}^{\theta_1} \quad (12)$$

由题意可知，龙头前把手的行进速度 v_0 始终为 1m/s ，由此可以求得龙头从初始时刻至 t 时刻沿螺线行进的路程，即该段阿基米德螺线的弧长：

$$l = v_0 t \quad (13)$$

初始时，龙头位于螺线第 16 圈 A 点处。龙头初始时刻极角 $\theta_0 = 16 \times 2\pi$ ，由此可得，基于弧长公式的坐标反解模型如下：

$$\begin{cases} L = b \left(\frac{1}{2} \theta \sqrt{\theta^2 + 1} + \frac{1}{2} \ln(\theta + \sqrt{\theta^2 + 1}) \right)_{\theta_0}^{\theta_t} \\ \theta_0 = 16 \times 2\pi \\ L = v_0 t \end{cases} \quad (14)$$

5.2.2 基于距离约束求解龙身、龙尾位置

根据题中板凳俯视图可以求得 $d_0 = (341 - 27.5 \times 2) \times 10^{-2} \text{m}$ 为龙头把手间的距离， $d_n = (220 - 27.5 \times 2) \times 10^{-2} \text{m}$ 为龙身、龙尾把手间的距离。

记初始时刻龙头前把手中心位置坐标为 (x_0, y_0) ，其余节前把手中心位置坐标 (x_n, y_n) ，当 $1 \leq n \leq 221$ 时为第 n 节龙身前把手中心位置坐标，当 $n=223$ 时为龙尾后把手中心位

置坐标。假设舞龙队在初始时刻前已按等距螺线排列，根据舞龙队在盘入过程中各把手均位于螺线上以及舞龙队间由长度不变的板凳相连可得距离约束为：

$$(x_{n+1} - x_n)^2 + (y_{n+1} - y_n)^2 = dn^2 \quad (15)$$

如下图所示，由于约束圆与等距螺线存在多个交点，即上述方程可能存在多解。根据各把手间的位置关系，正解应为沿螺线逆时针方向距离最短的交点。由此，可得到基于距离约束的龙身、龙尾位置求解模型如下：

$$\begin{aligned} & \arg \min(x_{n+1}, y_{n+1}) = \theta_{n+1} - \theta_n \\ s.t. & \begin{cases} (x_{n+1} - x_n)^2 + (y_{n+1} - y_n)^2 = dn^2 \\ x_n = b\theta_n \cos \theta_n \\ y_n = b\theta_n \sin \theta_n \\ \theta_0 = 16 \times 2\pi \\ b = 0.55 \\ \theta_{n+1} - \theta_n \geq 0 \end{cases} \end{aligned} \quad (16)$$

5.3 舞龙队速度求解模型

已知龙头前把手的行进速度始终保持 1m/s，即龙头前把手沿螺线的切线速度为 1m/s。如图所示，可将舞龙队各节的速度分别沿平行板凳方向和垂直板凳方向分解。根据同一节沿平行板凳方向速度相等可推导得到第 n 节板凳龙与第 $n-1$ 节板凳龙间的速度关系：

$$\begin{cases} v_{n-1} \cos \alpha_{n-1} = v'_{n-1} \\ v'_{n-1} = v'_n \\ v'_n = v_n \cos \alpha_n \end{cases} \quad (17)$$

上述方程联立可得：

$$v_n = \frac{v_{n-1} \cos \alpha_{n-1}}{\cos \alpha_n} \quad (18)$$

其中， α_{n-1} 为第 $n-1$ 、 n 节板凳龙前把手中心连线与第 $n-1$ 节板凳龙前把手中心处螺线切线夹角， α_n 为第 $n-1$ 、 n 节板凳龙前把手中心连线与第 n 节板凳龙前把手中心处螺线切线夹角。

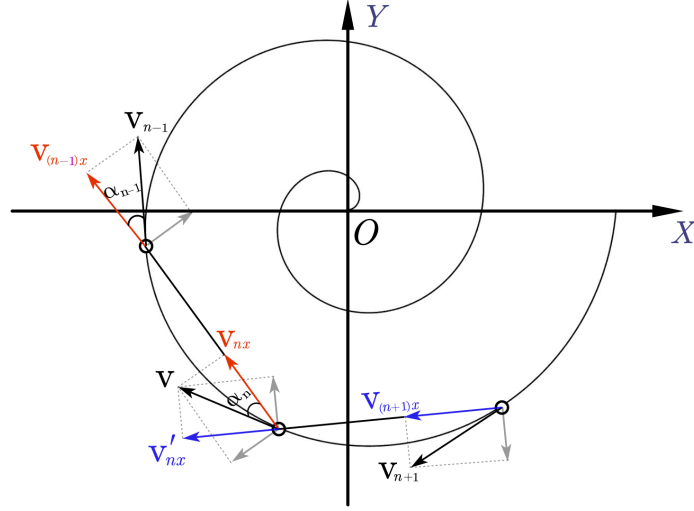


图 2 速度分解示意图

已知直角坐标系下阿基米德螺线方程为：

$$\begin{cases} x = b\theta \cos \theta \\ y = b\theta \sin \theta \end{cases} \quad (19)$$

其两边对 θ 求导：

$$\begin{cases} \frac{dx}{d\theta} = \frac{d}{d\theta}[b\theta \cos \theta] = b \cos \theta - b\theta \sin \theta \\ \frac{dy}{d\theta} = \frac{d}{d\theta}[b\theta \sin \theta] = b \sin \theta + b\theta \cos \theta \end{cases} \quad (20)$$

由此可得第 $n-1$ 节板凳龙前把手中心处螺线切线的方向向量为：

$$\vec{n}_1 = \left(\frac{dx}{d\theta}, \frac{dy}{d\theta} \right) = (b \cos \theta_{n-1} - b\theta_{n-1} \sin \theta_{n-1}, b \sin \theta_{n-1} + b\theta_{n-1} \cos \theta_{n-1}) \quad (21)$$

由前文求得的第 $n-1$ 、 n 节板凳龙前把手中心处的坐标可得其连线的方向向量为：

$$\vec{n}_2 = (x_n - x_{n-1}, y_n - y_{n-1})$$

综上，第 $n-1$ 、 n 节板凳龙前把手中心连线与第 $n-1$ 节板凳龙前把手中心处螺线切线夹角的余弦值为：

$$\cos \alpha_{n-1} = \frac{|\vec{n}_1 \cdot \vec{n}_2|}{|\vec{n}_1| \cdot |\vec{n}_2|} \quad (22)$$

同理可得，第 $n-1$ 、 n 节板凳龙前把手中心连线与第 n 节板凳龙前把手中心处螺线切线夹角的余弦值。

由此，可通过第 $n-1$ 节板凳龙的速度 v_{n-1} 求出第 n 节板凳龙的速度 v_n 。

5.4 模型求解

由于待求解的把手点位反解方程均是可微的且局部连续的，因此可用牛顿法进行求解，可以在较短的时间内求解得到高精度解。

牛顿法通过线性近似函数在某一点的局部行为，逐步逼近方程的根。其核心思想利用泰勒展开对函数 $f(x)$ 进行线性近似，并迭代更新以逼近解。假设 $f(x)$ 是在 x_n 处的可微函数，我们用它的一阶泰勒展开近似来表示 $f(x)$ 的局部行为：

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n) \quad (23)$$

为了求解非线性方程 $f(x)=0$ ，我们希望找到某个点 x_* ，使得在这个点上 $f(x_*) = 0$ 。我们从一个初始猜测值 x_0 出发，通过迭代逐步调整 x_n 使得 $f(x_n)$ 越来越接近 0。因此有迭代公式：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (24)$$

以上是传统的基于无约束问题的牛顿法，其存在对初始点敏感、需要计算导数、对多根问题的表现不佳、不能在求解中包含不等式约束等问题，因此本文提出以下四点改进措施：

① 割线近似切线

由于在求解过程中需要进行较多次的反解方程，因此通过利用差商近似导数，避免了在牛顿求解导数的过程，减少计算量。选取较小变化量 dx ，将 $x_n + dx$ 与 x_n 之间的割线近似为 x_n 处的导数，近似公式：

$$f'(x_n) \approx \frac{f(x_n + dx) - f(x_n)}{dx} \quad (25)$$

从而经过第一步改进的牛顿法迭代公式变为：

$$x_{n+1} = x_n - \frac{dx \cdot f(x_n)}{f(x_n + dx) - f(x_n)} \quad (26)$$

② 强制首步控制

由于模型中需要优化找到沿螺旋线距离最短的交点坐标，不是传统的无约束方程组问题，若直接采用牛顿法，则会产生多根问题而无法判断。因此对算法进行改进，使其解满足此目标。首先基于牛顿法对初始点敏感的情况，设置初始点为约束圆圆心，即 $x_0 = \theta_n$ ，以保证算法从一个合适的初始点开始搜索。

由于模型中需要优化找到沿螺旋线距离最短的交点坐标，不是传统的无约束方程组问题，若直接采用牛顿法，则会产生多根问题而无法判断。因此对算法进行改进，使其解满足此目标。首先基于牛顿法对初始点敏感的情况，设置初始点为约束圆圆心，即 $x_0 = \theta_n$ ，以保证算法从一个合适的初始点开始搜索。

此外，由于 $\theta_{n+1} - \theta_n > 0$ 的约束，方程的解不会在 θ_n 的负方向，因此在算法第一步时强制控制其向正方向进行搜索，以保证不会出现持续向负方向搜索的情况。

同时，该反解方程的解集较为密集，为了防止算法首步搜索步长过大而错过距离最短的交点坐标，为首步搜索步长设置上限 p ，以保证解的准确。

从而经过第二步改进的牛顿法迭代公式变为：

$$\begin{cases} x_0 = \theta_n \\ x_2 = x_1 + \min(|\frac{dx f(x)}{f(x_1 + dx) - f(x_1)}|, p) \\ x_{n+1} = x_n - \frac{dx f(x)}{f(x_n + dx) - f(x_n)} \end{cases} \quad (27)$$

③ 小步长搜索

同样为了防止算法首步搜索步长过大而错过距离最短的交点坐标，对算法搜索步长进行系数限制，以减少搜索振荡带来的影响。

从而经过第三步改进的牛顿法迭代公式变为：

$$\begin{cases} x_0 = \theta_n \\ x_2 = x_1 + \min(|\frac{dx f(x)}{f(x_1 + dx) - f(x_1)}|, p) \\ x_{n+1} = x_n - h \frac{dx f(x)}{f(x_n + dx) - f(x_n)} \end{cases} \quad (28)$$

④ 负向重搜机制

由于该方程在正向与负向均有解，在进行前几步改进后，算法仍有可能搜索到负向的解，其很大原因是初始点距离负向较近，与正向解相距较远。因此引入负向重搜机制，当方程解处于初始点负向时，适当扩大初始点，进行重新搜索，直到找到正向解为止。

从而经过第四步改进的牛顿法迭代公式变为：

$$\begin{aligned} & \mathbf{do}\{x_0 = 1.1^k \theta_n (k \text{ 为重搜次数}) \\ & \quad x_1 = x_1 + \min(|\frac{dx f(x)}{f(x_1 + dx) - f(x_1)}|, p) \\ & \quad \quad \quad \vdots \\ & \quad x_{n+1} = x_n - hx \frac{dx f(x_n)}{f(x_n + dx) - d(x_n)}\} \\ & \mathbf{while}\{x_{n+1} > \theta_n\} \end{aligned} \quad (29)$$

5.5 结果与分析

在初始时刻到 300s 的范围内，使用改进牛顿法求解位置反解方程与速度迭代方程得到部分结果如下表所示：

表 1 位置结果

	0s	60s	120s	180s	240s	300s
龙头 x(m)	8.800000	5.799209	-4.084887	-2.963609v	2.594494	4.420274
龙头 y(m)	0.000000	-5.771092	-6.304479	6.094780	-5.356743	2.320429
第 1 节龙身 x(m)	8.363824	7.456758	-1.445473	-5.237118	4.821221	2.459488
第 1 节龙身 y(m)	2.826544	-3.440399	-7.405882	4.359627	-3.561949	4.402476
第 51 节龙身 x(m)	-9.518732	-8.686317	-5.543149	2.890456	5.980011	-6.301346
第 51 节龙身 y(m)	1.341137	2.540108	6.377946	7.249289	-3.827759	0.465830
第 101 节龙身 x(m)	2.913983	5.687115	5.361938	1.898794	-4.917372	-6.237722
第 101 节龙身 y(m)	-9.918311	-8.001384	-7.557639	-8.471614	-6.379874	3.936009
第 151 节龙身 x(m)	10.861726	6.682312	2.388758	1.005154	2.965379	7.040741
第 151 节龙身 y(m)	1.828753	8.134544	9.727411	9.424751	8.399720	4.393012
第 201 节龙身 x(m)	4.555103	-6.619663	-10.627210	-9.287720	-7.457152	-7.458663
第 201 节龙身 y(m)	10.725118	9.025571	1.359848	-4.246672	-6.180726	-5.263383
龙尾（后）x(m)	-5.305444	7.364557	10.974348	7.383896	3.241052	1.785034
龙尾（后）y(m)	-10.676584	-8.797992	0.843472	7.492370	9.469336	9.301164

表 2 速度结果

	0s	60s	120s	180s	240s	300s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999971	0.999961	0.999945	0.999917	0.999859	0.999709
第 51 节龙身 (m/s)	0.999742	0.999662	0.999538	0.999331	0.998941	0.998065
第 101 节龙身 (m/s)	0.999575	0.999453	0.999269	0.998971	0.998435	0.997302
第 151 节龙身 (m/s)	0.999448	0.999299	0.999078	0.998727	0.998115	0.996861
第 201 节龙身 (m/s)	0.999348	0.999180	0.998935	0.998551	0.997894	0.996574
龙尾（后）(m/s)	0.999311	0.999136	0.998883	0.998489	0.997816	0.996478

固定时间为 300s，作出各把手的速度变化曲线如下图 1 所示。再对于龙尾后把手，作出其在 [0s,300s] 内的速度变化曲线如下图 2 所示。

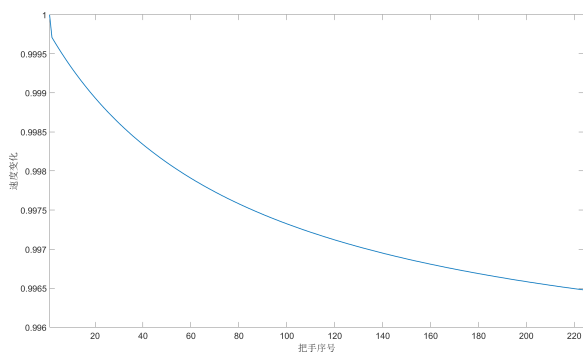


图 3 300s 时各把手速度变化情况

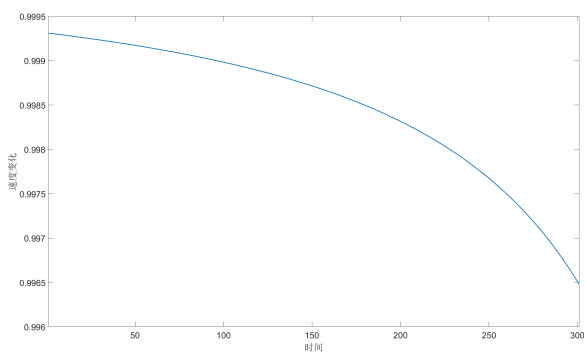


图 4 龙尾速度相对于时间的变化情况

可以看出对于固定时间下，把手距离龙头越远，其速度相对于龙头速度减小的越多，这种减小随着距离的增加而趋于稳定，呈现一种渐进稳定的态势。

而对于龙尾速度关于时间情况可以看到龙尾速度随着行进时间的增加而逐渐减小，同时其减小率随着时间的增长而逐步增加。

5.5.1 灵敏度分析

将龙尾后把手作为研究对象，研究在 0s 时，螺距变化与龙头初速度对其速度的影响如下图所示：

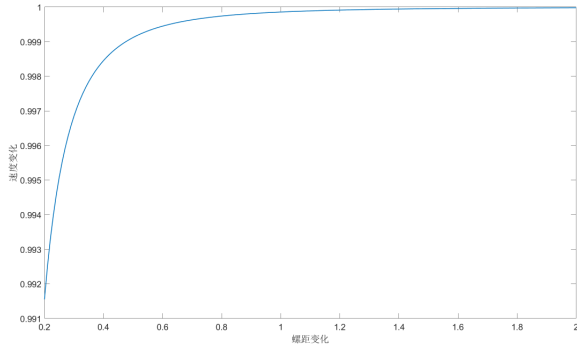


图 5 龙尾关于螺距变化的速度变化

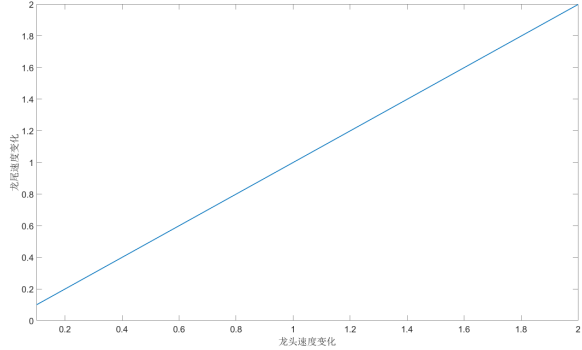


图 6 龙尾关于龙头初速度变化情况

在螺距变化的过程中，龙尾后把手速度增长呈现一种先快后慢的趋势，最终趋近于龙首初速度，本文认为由于螺距增大，导致整体舞龙队所在曲率变小，导致速度衰减变小。

而在龙头初速度变化过程中，龙尾后把手速度变化呈现明显的线性关系，进行分析可知当舞龙队中各把手位置保持不变时，两两之间的速度比值不变，因此可知龙尾后把手与龙头初速度比值不变。

六、第二问模型的建立与求解

6.1 问题二模型的建立

本问要求舞龙队以问题一设定的螺线盘入轨迹行进，在板凳之间不发生碰撞的约束下，求解舞龙队盘入的终止时刻，并给出此时舞龙队的位置和速度。为探究各节板凳龙中存在板凳相碰的情况，首先需根据问题一中得到的各把手中心位置信息求出各板凳龙四周顶点坐标；然后建立板凳碰撞判断模型，在不发生碰撞的约束条件下求解终止时刻。

6.1.1 基于方向向量的板凳龙四周顶点计算模型

为便于后续研究板凳之间碰撞情况，将板凳龙首尾相连简化为 223 个矩形在把手中心处相连，其中龙头矩形长 3.41m，宽 0.3m，龙身、龙尾矩形长 2.2m，宽 0.3m。通过问题一可以求得 t 时刻第 n-1 与第 n 节板凳龙前把手中心位置 (x_{n-1}, y_{n-1}) ， (x_n, y_n) 。

根据几何关系可得第 n-1 个矩形长边的方向向量 $\vec{n}_1$ 为：

$$\vec{n}_1 = (x_n - x_{n-1}, y_n - y_{n-1}) \quad (30)$$

设向量 $\vec{n}_3$ 为该长边的方向向量的法向量，由垂直关系 $\vec{n}_1 \cdot \vec{n}_3 = 0$ ，可得法向量 $\vec{n}_3$ ，即第 $n-1$ 个矩形短边的方向向量为：

$$\vec{n}_3 = (y_{n-1} - y_n, x_n - x_{n-1}) \quad (31)$$

根据题中龙头、龙身、龙尾俯视图中尺寸可得各矩形四周顶点距离前、后把手中心在 $\vec{n}_1$ 、 $\vec{n}_3$ 方向上的长度 p 、 q 分别为：

$$p = 27.5 \times 10^{-2}$$

$$q = 15 \times 10^{-2}$$

如图 7 所示，根据上述的分析，可求得各矩形四周顶点（顺时针依次为 A、B、C、D）为：

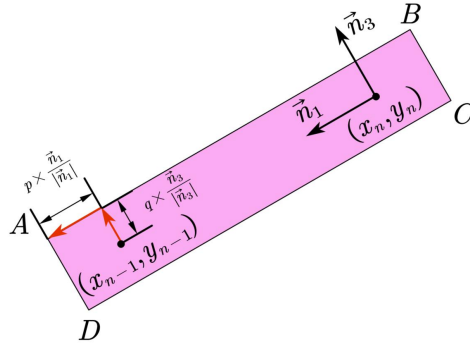


图 7 矩形四周顶点求解图

$$\left\{ \begin{array}{l} (x_{na}, y_{na}) = (x_{n-1}, y_{n-1}) - q \times \frac{\vec{n}_3}{|\vec{n}_3|} + p \times \frac{\vec{n}_1}{|\vec{n}_1|} \\ (x_{nb}, y_{nb}) = (x_{n-1}, y_{n-1}) + q \times \frac{\vec{n}_3}{|\vec{n}_3|} + p \times \frac{\vec{n}_1}{|\vec{n}_1|} \\ (x_{nc}, y_{nc}) = (x_n, y_n) + q \times \frac{\vec{n}_3}{|\vec{n}_3|} - p \times \frac{\vec{n}_1}{|\vec{n}_1|} \\ (x_{nd}, y_{nd}) = (x_n, y_n) - q \times \frac{\vec{n}_3}{|\vec{n}_3|} - p \times \frac{\vec{n}_1}{|\vec{n}_1|} \end{array} \right. \quad (32)$$

6.1.2 板凳碰撞判断模型

将板凳龙简化为矩形相连后，板凳之间不发生碰撞约束即可转化为第 i 个矩形与其相连矩形外的任意矩形 j 在运动平面中不发生重叠。则将该约束条件转变为矩形 i 内的点集 P_i 不同时存在于矩形 j 中，即：

$$\left\{ \begin{array}{l} P_i \cap P_j = \phi \\ 0 \leq i < j \leq n \\ j - i > 1 \end{array} \right. \quad (33)$$

分离轴定理由 Hermann Minkowski 提出，可用于解决凸多边形碰撞问题，该理论表

明：如果存在且仅需要存在一条轴线，使得两个凸面物体在该轴上的投影没有重叠，那么这两个凸面物体就没有重叠，而这个轴线称为分离轴。相反而言，如果两个凸面体存在重叠区域，那么它们在任何一个投影轴上的投影都会重合。

Step1: 计算分离轴向量

分离轴定理同时指出：对于凸形状可能的分离轴总是垂直于图形的边缘。因此选取两个矩形各边的法线方向作为潜在分离轴进行检验。由于矩形的对边平行性，分离轴仅需为每个矩形的两条直角边对应的法向量。对于矩阵 **A**、**B**，其各定点可分别表示为 $A_i(x_{Ai}, y_{Ai})$ 与 $B_i(x_{Bi}, y_{Bi})$ ($i = 1 \sim 4$ ，表示顺时针时矩阵各定点编号)，则矩阵上两条直角边的方向向量为：

$$\begin{cases} \vec{v}_1 = (x_{A2} - x_{A1}, y_{A2} - y_{A1}) \\ \vec{v}_2 = (x_{A3} - x_{A2}, y_{A3} - y_{A2}) \\ \vec{v}_3 = (x_{B2} - x_{B1}, y_{B2} - y_{B1}) \\ \vec{v}_4 = (x_{B3} - x_{B2}, y_{B3} - y_{B2}) \end{cases} \quad (34)$$

进而可根据公式 $\vec{n} = (-v_y, v_x)$ 求得各边法向量为：

$$\begin{cases} A_{j \min} = \min[P_{A,1,j}, P_{A,2,j}, P_{A,3,j}, P_{A,4,j}] \\ A_{j \max} = \max[P_{A,1,j}, P_{A,2,j}, P_{A,3,j}, P_{A,4,j}] \\ B_{j \min} = \min[P_{B,1,j}, P_{B,2,j}, P_{B,3,j}, P_{B,4,j}] \\ B_{j \max} = \max[P_{B,1,j}, P_{B,2,j}, P_{B,3,j}, P_{B,4,j}] \end{cases} \quad (35)$$

Step2: 矩阵顶点在分离轴上的投影

确定分离轴后，要将两个矩形的所有顶点沿各分离轴方向进行投影。对于每一个顶点 $P(x, y)$ 和给定的分离轴方向向量 $\vec{n} = (-v_y, v_x)$ ，投影值 **P** 可以通过点积公式计算：

$$T = P \cdot \vec{n} \quad (36)$$

Step3: 顶点投影区域范围

对矩形 **A** 和 **B** 的所有顶点进行投影，可得到每个矩形在各分离轴上的投影范围 $[A_j \min, A_j \max]$ ，矩形 **B** 在同一轴上的投影范围是 $[B_j \min, B_j \max]$ ($j = 1 \sim 4$ ，表示分离轴序号)，因此投影范围公式有：

$$\begin{cases} T_{Aj} = [\min_i(P_{Ai} \cdot n_j), \max_i(P_{Ai} \cdot n_j)] \\ T_{Bj} = [\min_i(P_{Bi} \cdot n_j), \max_i(P_{Bi} \cdot n_j)] \end{cases} \quad (37)$$

Step4: 重叠判断

在每条分离轴上，要检查两个矩形的投影是否重叠，则可比较两个矩形的投影范围是否重叠。因此可以得到不重叠判别公式有：

$$\exists j, [\max(T_{Aj}) \leq \min(T_{Bj})] \cup [\min(T_{Aj}) \geq \max(T_{Bj})] \quad (38)$$

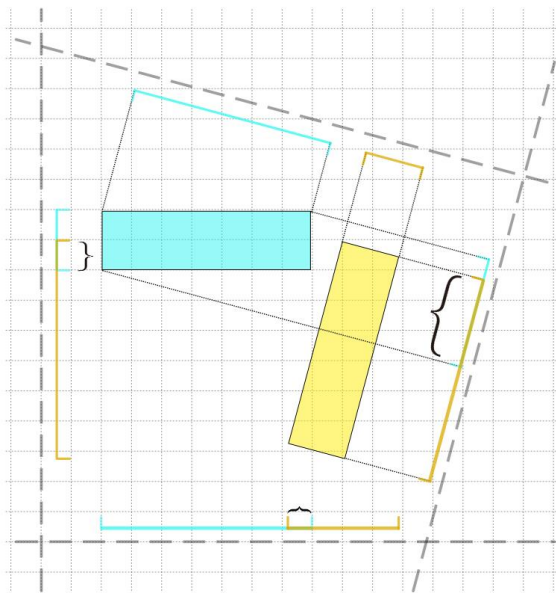


图 8 无重叠区域

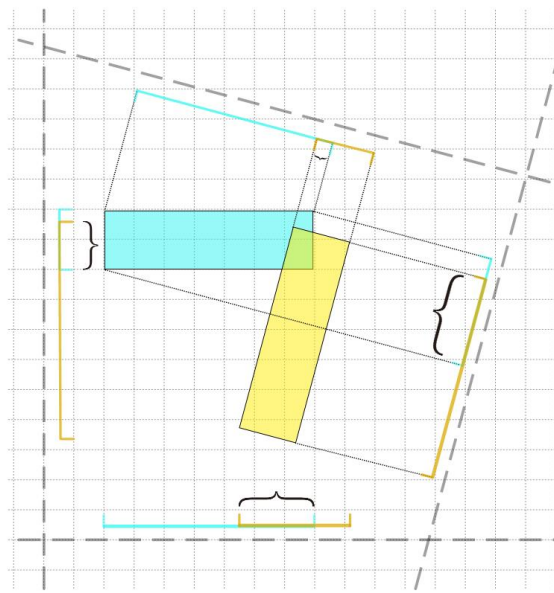


图 9 有重叠区域

举例而言，对于如上左图，两矩形虽在左右下三条分离轴上有重合区域，但其在上方的分离轴上没有重合区域，因此其在二维平面中没有重合区域。而对于如上右图，两矩形在四条分离轴上均有重合区域，从而其在二维平面中重合。

6.2 模型算法

6.2.1 粗细网格算法求解

Step1: 重叠判断

通过改进牛顿算法与分离轴判别准则判断碰撞关系，由于各龙身板凳形状均相同，因此其运动具有历史相同性。对于龙身而言，可以只需判断第一个龙身与其他板凳的碰撞关系，同时由于龙头与各龙身尺寸均不相同，也需判断龙头与其他板凳的碰撞关系。由于碰撞仅存在于与外围板凳中，控制碰撞对象的极角与判断对象的极角之差小于 4π 即可，可以大大简化计算量。

Step2: 初步选取大步长

对于该螺旋线总长为 442.59m，因此选取大步长遍历范围为 [0m,442m]，遍历步长为 1m，在遍历过程中判断碰撞关系，若为碰撞则直接退出遍历，并记录此时的结果。

Step3: 缩短搜索步长

对于初步的大步长求解得到结果为 412m，缩短搜索步长为 0.001m，范围控制为 [411m,413m]，进行小步长遍历求解。

Step4: 遍历搜索结果

经过粗细网格遍历后求解得到结果为 412.474m，即终止时刻为 412.474s。

6.2.2 灵敏度分析

对于板长与板宽进行灵敏度分析，得到结果如下图所示：

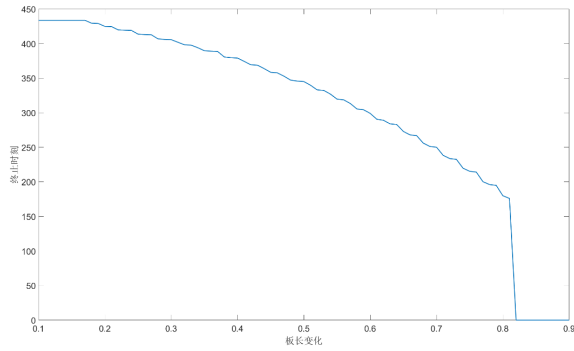


图 10 终止时刻对于板长变化情况

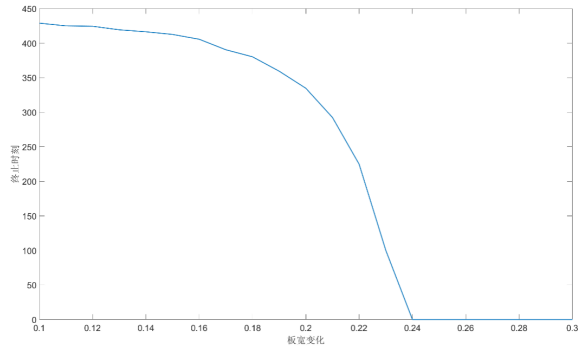


图 11 终止时刻对于板宽变化情况

对于板长的灵敏度变化，存在明显的阶梯型变化，本文认为在阶梯型的平整处限制终止时刻的主要因素应为板宽，而对于阶梯型的斜坡处限制终止时刻的主要因素应为板长。而对于板宽的灵敏度分析中，没有明显的阶梯型变化。此外无论是板长还是板宽，均存在变化上限，此上限应为在初始时刻板凳尺寸过大而导致碰撞。

表 3 速度第二问结果

	龙头	第 1 节龙身	第 51 节龙身	第 101 节龙身	第 151 节龙身	第 201 节龙身	龙尾（后）
x(m)	1.206751	0.000000	1.277639	-0.532541	0.972532	-7.892628	0.952527
y(m)	1.944933	1.750897	4.327717	-5.880530	-6.957011	-1.234445	8.323198
v(m/s)	1.000000	0.991553	0.976863	0.974555	0.973613	0.973101	0.972943

七、第三问模型的建立与求解

7.1 问题三模型的建立

本文要求在龙头前把手能够沿着螺线盘入到调头空间的条件下，求解舞龙队盘入螺线的最小螺距。根据问题二分析可得，舞龙队越靠近螺线中心，板凳之间越容易发生碰撞。为使得龙头前把手顺利盘入调头空间，且确保板凳之间不发生碰撞，需对不同螺距下龙头前把手进入调头空间时各板凳间碰撞情况进行判断，从而确定最小螺距。

7.1.1 最小螺距求解模型

当龙头前把手进入调头空间时，为进行板凳碰撞判断，需对此时整个舞龙队的位置进行求解。假设舞龙队沿螺距为 b 的等距螺线顺时针盘入，根据调头圆形区域直径为 $9m$ ，代入阿基米德螺线方程有： $4.5 = 6\theta$ ，可解得此时 $\theta = \frac{4.5}{b}$ ，则龙头前把手的坐标为：

$$\begin{aligned}
 (x_0, y_0) &= (b\theta \cos \theta, b\theta \sin \theta) \\
 &= \left(b \frac{4.5}{b} \cos \frac{4.5}{b}, b \frac{4.5}{b} \sin \frac{4.5}{b}\right) \\
 &= \left(4.5 \cos \frac{4.5}{b}, 4.5 \sin \frac{4.5}{b}\right)
 \end{aligned} \tag{39}$$

根据问题一分析可得，此时整个舞龙队的位置可由如下位置求解模型求得：

$$\begin{aligned} \text{ang min}(x_{n+1}, y_{n+1}) &= \theta_{n+1} - \theta_n \\ \text{s.t.} \quad &\begin{cases} (x_{n+1} - x_n)^2 + (y_{n+1} - y_n)^2 = d_n^2 \\ x_{n+1} = b\theta_{n+1} \cos \theta_{n+1} \\ y_{n+1} = b\theta_{n+1} \sin \theta_{n+1} \\ \theta_{n+1} - \theta_n > 0 \\ x_0 = 4.5 \cos \frac{4.5}{b} \\ y_0 = 4.5 \sin \frac{4.5}{b} \end{cases} \end{aligned} \quad (40)$$

通过上述模型得到各节板凳龙把手位置坐标后，根据问题二中基于方向向量的板凳龙四周顶点计算模型，可求解各板凳顶点坐标。

基于上述分析，结合问题二中的板凳碰撞判断模型可建立最小螺距求解模型。

7.2 问题三结果分析

本问同样采用粗细网格搜索算法，第一次大步长设置为1cm，搜索范围为[170cm,0cm]，由于在碰撞时需要退出遍历，因此从后向前搜索，得到第一次大步长遍历结果为41cm。第二次小步长设置为0.001cm，搜索范围为[42cm,40cm]，得到结果为41.713cm。并作出此时的空间示意图如下所示，此时龙头板凳正好与外围一个板凳向切，符合最小化螺距的目标。

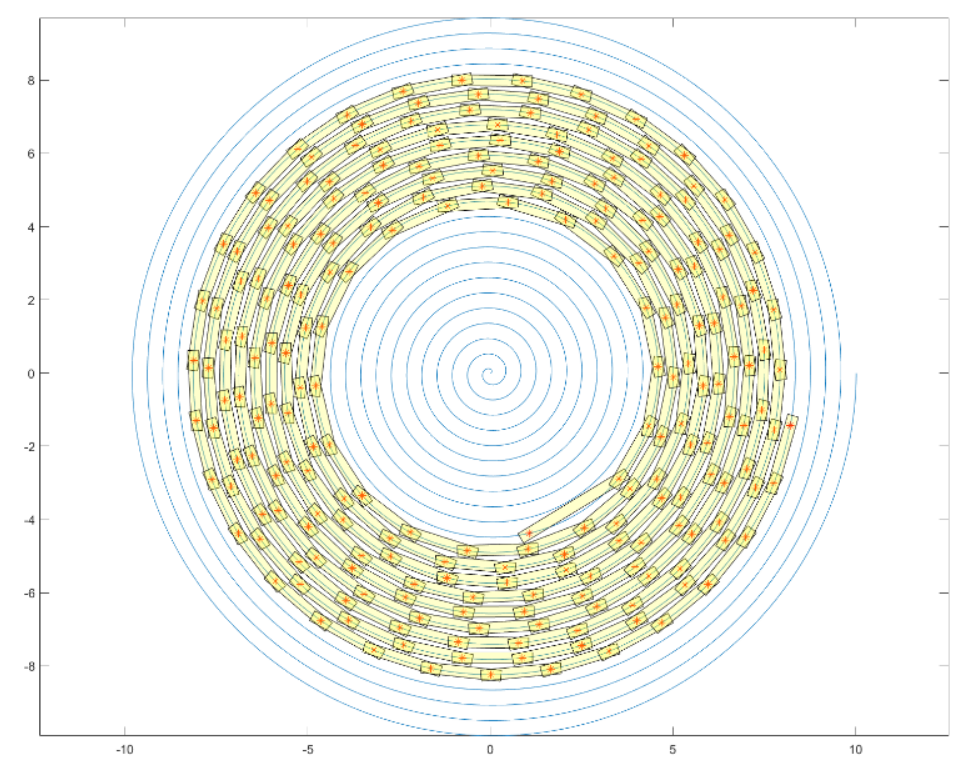


图 12 板凳龙排列示意图

八、问题四模型的建立与求解

8.1 问题四模型的建立

由问题背景可知，本题的核心为目标优化问题，在保持调头路径各部分相切的条件下，通过调整圆弧，实现调头曲线尽可能短的目标。在前文基础上，本第四问重点讨论满足题意所存在的约束条件，最终建立调头曲线优化模型。

已知盘入螺线在极坐标系与直角坐标系下的方程分别为： $r = b\theta$ 和 $\begin{cases} x = b\theta \cos \theta \\ y = b\theta \sin \theta \end{cases}$ ，

由盘入螺线与盘出螺线关于螺线中心即原点呈中心对称，可以求得盘出螺线在极坐标系与直角坐标系下的方程分别为：

$$r = b(-\theta) = -b\theta \quad (\theta > 0) \quad (41)$$

$$\begin{cases} x = -b\theta \cos \theta \\ y = -b\theta \sin \theta \end{cases} \quad (\theta > 0) \quad (42)$$

决策变量：

根据题中所给信息，调头路径由两段相切的圆弧连接而成，前一段圆弧的半径是后一段的 2 倍，且它与盘入、盘出螺线均相切。根据几何分析可知，确定从盘入螺线进入调头圆弧的切入点与从调头圆弧进入盘出螺线的切出点后，S 形调头路径即可确定。因此，本文以切入点 M 的极角 θ_M 与切出点 N 的极角 θ_N 作为决策变量。

进一步，结合题目背景将其转化为数学形式的约束条件。

目标函数：

通过构建优化目标以及前文几何分析，在调头区域内调头曲线包含调头圆弧 1 与调头圆弧 2，由此本文得到了以下目标函数：

$$\min S = (\theta_{o1 \max} - \theta_{o1 \min}) \cdot 2r + (\theta_{o2 \max} - \theta_{o1 \min}) \cdot r \quad (43)$$

约束条件：

舞龙队的调头曲线受到调头空间范围、各部分相切、调头圆弧与盘入螺线缠绕、板凳之间碰撞情况的协同限制，本文基于此得到了以下约束条件：

● **调头空间范围约束：**

调头空间是以螺线中心为圆心、直径为 9m 的圆形区域。舞龙队需在该空间内完成调头，即切入与切出的极径绝对值需小于圆形区域的半径。则对于 θ_M 、 θ_N 有以下约束：

$$\begin{cases} \theta_M < \frac{r}{b} \\ \theta_N < \frac{r}{b} \end{cases} \quad (44)$$

● **盘出螺线与盘入曲线中心对称约束：**

根据题目背景，舞龙队在行进过程中盘出螺线与盘入曲线需保持关于螺线中心呈中心对称。由此调头空间内切入点与切出点关于原点对称，否则舞龙队在盘入、盘出过程中其行进路径形成的螺线轨迹不中心对称。综上可得切入点与切出点有以下约束：

$$\theta_M = \theta_N \quad (45)$$

• 调头曲线各部分相切约束：

两段圆弧相切连接而成的 S 形调头路径中，前一段圆弧的半径是后一段半径的 2 倍。设与盘出螺线相连的调头圆弧圆心为 O_2 ，半径为 r ，与盘入螺线相连的调头圆弧圆心为 O_1 ，半径为 $2r$ 。

由前文可得盘入螺线的切向量为：

$$\vec{n}_1 = \left(\frac{dx}{d\theta}, \frac{dy}{d\theta} \right) = (b \cos \theta - b\theta \sin \theta, b \sin \theta + b\theta \cos \theta_{n-1})$$

设盘入螺线的径向法向量为 $\vec{N}_1$ ，由垂直关系可得：

$$\vec{N}_1 = \left(-\frac{dy}{d\theta}, \frac{dx}{d\theta} \right) = (-(b \sin \theta + b\theta \cos \theta), b \cos \theta - b\theta \sin \theta) \quad (46)$$

同理可得盘出螺线的径向法向量 N_2 为：

$$\vec{N}_2 = \left(-\frac{dy}{d\theta}, \frac{dx}{d\theta} \right) = (b\theta \cos \theta + b \sin \theta, b\theta \sin \theta - b \cos \theta) \quad (47)$$

记切入点 M 与切出点 N 的坐标分别为 (x_M, y_M) 、 (x_N, y_N) ，由 S 形调头路径中两段圆弧与盘入、盘出螺线相切，可以求得这两段圆弧的圆心坐标分别为：

$$(x_{o1}, y_{o1}) = (x_M, y_M) + \frac{\vec{N}_1}{|\vec{N}_1|} \cdot 2r \quad (48)$$

$$(x_{o2}, y_{o2}) = (x_N, y_N) + \frac{\vec{N}_2}{|\vec{N}_2|} \cdot r \quad (49)$$

两段圆弧相切，则有两圆心间距离 d_{o1o2} 为 $3r$ ，即：

$$\sqrt{(x_{o1} - x_{o2})^2 + (y_{o1} - y_{o2})^2} = 3r \quad (50)$$

通过两圆心坐标以及两段圆弧圆心间距离的比例关系，可以求得这两段圆弧切点 R 的坐标：

$$(x_R, y_R) = (x_{o1}, y_{o1}) + \frac{2}{3}d_{o1o2} \quad (51)$$

• 板凳运动中的刚体约束：

如图（13）所示，假设板凳为不发生形变的刚体，其在曲线移动过程中可能产生上述“卡住”情况。本文将基于速度分解在以下篇幅讨论避免板凳龙在行径过程中产生卡位限制。

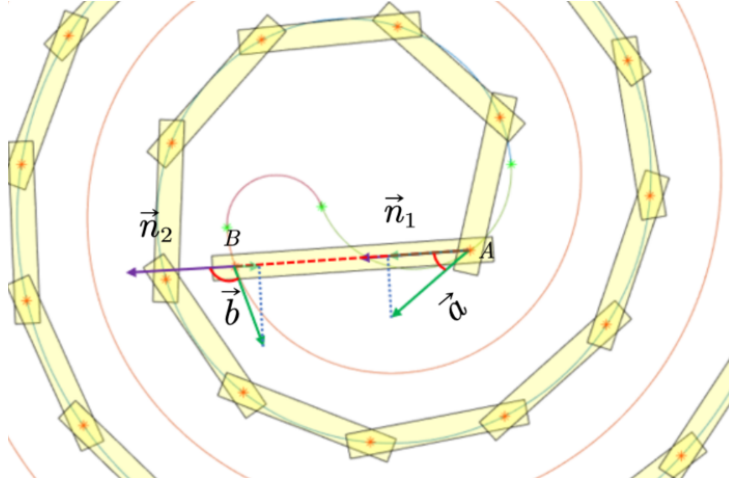


图 13 刚体约束示意图

图中所示为第 $n-1$ 节该凳龙，记其前后把手分别为上 A、B 两点，该两点处的切向量分别为 $\vec{a}$ 、 $\vec{b}$ 。该两点的切向量向板凳方向向量 $\vec{n}_1$ 投影，可得切向量 $\vec{a}$ 、 $\vec{b}$ 的投影分别为：

$$Proj_{\vec{n}_1} \vec{a} = \frac{\vec{a} \cdot \vec{n}_1}{|\vec{n}_1|} \quad (52)$$

$$Proj_{\vec{n}_1} \vec{b} = \frac{\vec{b} \cdot \vec{n}_1}{|\vec{n}_1|} \quad (53)$$

根据题目背景分析可知，板凳龙 A、B 两点处沿板凳方向的速度方向即为切向量 $\vec{a}$ 、 $\vec{b}$ 向板凳方向向量 $\vec{n}_1$ 的投影。若进行正常运动，则 A、B 两点沿板凳方向的分速度应沿同一方向，即有以下约束：

$$\cos \langle Proj_{\vec{n}_1} \vec{a}, Proj_{\vec{n}_1} \vec{b} \rangle > 0 \quad (54)$$

• 板凳之间碰撞情况约束：

研究舞龙队在调头空间内调头过程中板凳之间的碰撞情况，首先需要根据龙头在该范围内的行进过程确定整队中其余板凳的位置，然后利用基于方向向量的板凳龙四周顶点计算模型将板凳龙简化为矩形相连，最后根据板凳碰撞判断模型得到板凳之间碰撞情况约束。

调头曲线参数方程

调头空间内舞龙队的行进路径包含盘入螺线段、与盘出螺线相连的调头圆弧 1、与盘入螺线相连的调头圆弧 2 以及盘出螺线段。他们在直角坐标系下的参数方程求解过程如下：

(1) 盘入螺线段：

根据调头空间范围以及切入点 M 极角可以求得盘入螺线段的定义域为： $\theta_0 < \theta < \theta_M$ 则其参数方程为：

$$\begin{cases} x = b\theta \cos \theta \\ y = b\theta \sin \theta \\ \theta_M \leq \theta \leq \theta_0 \end{cases} \quad (55)$$

(2) 调头圆弧 1:

对调头圆弧 1 的两个端点切入点 M 与切点 R 坐标进行坐标变换, 然后使用三角函数对其定义域进行求解:

$$\begin{cases} (\Delta x_M, \Delta y_M) &= (x_M, y_M) - (x_{o1}, y_{o1}) \\ \theta_{o1} &= \arctan(\frac{\Delta y_M}{\Delta x_M}) \end{cases} \quad (56)$$

$$\begin{cases} (\Delta x_R, \Delta y_R) &= (x_R, y_R) - (x_{o1}, y_{o1}) \\ \theta'_{o1} &= \arctan(\frac{\Delta y_R}{\Delta x_R}) \end{cases} \quad (57)$$

由于 $\arctan$ 函数求得的角度范围为 $(-\pi/2, \pi/2)$, 其求得结果无法区分象限, 因此还需通过圆弧旋转方向判断所求角度的取值:

$$\theta = \begin{cases} 2\pi - \arctan(\frac{\Delta y}{\Delta x}), \arctan(\frac{\Delta y}{\Delta x}) > 0 \\ -\arctan(\frac{\Delta y}{\Delta x}), \arctan(\frac{\Delta y}{\Delta x}) < 0 \end{cases} \quad (58)$$

通过上式对 θ_{o1} 、 θ'_{o1} 修正可得该调头圆弧的定义域为 $\theta_{o1 \min} \leq \theta_{o1} \leq \theta_{o1 \max}$ 则其参数方程为:

$$\begin{cases} x = x_{o1} + 2r \cos \theta_{o1} \\ y = y_{o1} + 2r \sin \theta_{o1} \\ \theta_{o1 \min} \leq \theta_{o1} \leq \theta_{o1 \max} \end{cases} \quad (59)$$

(3) 调头圆弧 2:

对调头圆弧 2 的两个端点切入点 N 与切点 R 坐标进行坐标变换, 然后使用三角函数对其定义域进行求解:

$$\begin{cases} (\Delta x_N, \Delta y_N) &= (x_N, y_N) - (x_{o2}, y_{o2}) \\ \theta_{o2} &= \arctan(\frac{\Delta y_N}{\Delta x_N}) \end{cases} \quad (60)$$

$$\begin{cases} (\Delta x'_R, \Delta y'_R) &= (x_R, y_R) - (x_{o2}, y_{o2}) \\ \theta'_{o2} &= \arctan(\frac{\Delta y'_R}{\Delta x'_R}) \end{cases} \quad (61)$$

同理, 根据下式判断所求角度的取值:

$$\theta = \begin{cases} \arctan(\frac{\Delta y}{\Delta x}), \arctan(\frac{\Delta y}{\Delta x}) > 0 \\ 2\pi + \arctan(\frac{\Delta y}{\Delta x}), \arctan(\frac{\Delta y}{\Delta x}) < 0 \end{cases} \quad (62)$$

通过上式对 θ_{o2} 、 θ'_{o2} 修正可得该调头圆弧的定义域为 $\theta_{o2 \min} \leq \theta_{o2} \leq \theta_{o2 \max}$ 则其参数方程为:

$$\begin{cases} x = x_{o2} + r \cos \theta_{o2} \\ y = y_{o2} + r \sin \theta_{o2} \\ \theta_{o2 \min} \leq \theta_{o2} \leq \theta_{o2 \max} \end{cases} \quad (63)$$

(4) 盘出螺线段:

根据调头空间范围以及切出点N极角可以求得盘出螺线段的定义域为: $\theta_N \leq \theta \leq \theta'$, 则其参数方程为:

$$\begin{cases} x = -b\theta \cos \theta \\ y = -b\theta \sin \theta \\ \theta_N \leq \theta \leq \theta' \end{cases} \quad (64)$$

根据问题一中舞龙队位置求解模型的分析, 板凳前、后把手间的距离约束即为第 n 节板凳前把手位于以第 $n-1$ 节板凳前把手位置为圆心, d_{n-1} 为圆心的圆上。为了与该调头区域内曲线的参数表达形式统一, 该约束圆的参数方程形式为:

$$\begin{cases} x = x_{n-1} + d_{n-1} \cos \theta \\ y = y_{n-1} + d_{n-1} \sin \theta \end{cases} \quad (65)$$

舞龙队位置模型

舞龙队在调头空间内存在以上四段曲线, 基于此本文中舞龙队位置求解模型需分为以下四种情况:

- ① 当第 $n-1$ 节板凳前把手位于**盘入螺线段**时, 为求解第 n 节板凳前把手需将盘入螺线段方程 (55) 与约束圆方程 (65) 联立, 根据前文所分析的多解判断条件, 得到舞龙队位置求解模型如下:

$$\begin{aligned} (x_n, y_n) &= \arg \min(\theta_n - \theta_{n-1}) \\ \begin{cases} x = b\theta \cos \theta \\ y = b\theta \sin \theta \\ x = x_{n-1} + d_{n-1} \cos \theta \\ y = y_{n-1} + d_{n-1} \sin \theta \\ x_{n-1} = b\theta_{n-1} \cos \theta_{n-1} \\ y_{n-1} = b\theta_{n-1} \sin \theta_{n-1} \\ \theta_n - \theta_{n-1} > 0 \\ \theta_M \leq \theta \leq \theta_0 \end{cases} \end{aligned} \quad (66)$$

- ② 当第 $n-1$ 节板凳前把手位于**调头圆弧 1** 时, 根据前一情况分析, 可得调头圆弧 1 方

程（59）与约束圆方程（65）联立得到舞龙队位置求解模型如下：

$$(x_n, y_n) = \arg \min(\theta_n - \theta_{n-1})$$

$$\begin{cases} x = x_{o1} + 2r \cos \theta_{o1} \\ y = y_{o1} + 2r \sin \theta_{o1} \\ x = x_{n-1} + d_{n-1} \cos \theta \\ y = y_{n-1} + d_{n-1} \sin \theta \\ x_{n-1} = b\theta_{n-1} \cos \theta_{n-1} \\ y_{n-1} = b\theta_{n-1} \sin \theta_{n-1} \\ \theta_n - \theta_{n-1} > 0 \\ \theta_{o1 \min} \leq \theta_{o1} \leq \theta_{o1 \max} \end{cases} \quad (67)$$

若上述联立方程无解，则说明第 n 节板凳前把手位于盘入螺线段，此时需将盘入螺线段方程（55）与约束圆方程（65）联立，由于盘入螺线段的范围约束，目标函数应更改为 $(x_n, y_n) = \arg \min(\theta_n - \theta_M)$ ，得到舞龙队位置求解模型如下：

$$(x_n, y_n) = \arg \min(\theta_n - \theta_M)$$

$$\begin{cases} x = x_{o1} + 2r \cos \theta_{o1} \\ y = y_{o1} + 2r \sin \theta_{o1} \\ x = x_{n-1} + d_{n-1} \cos \theta \\ y = y_{n-1} + d_{n-1} \sin \theta \\ x_{n-1} = b\theta_{n-1} \cos \theta_{n-1} \\ y_{n-1} = b\theta_{n-1} \sin \theta_{n-1} \\ \theta_n - \theta_M > 0 \\ \theta_{o1 \min} \leq \theta_{o1} \leq \theta_{o1 \max} \end{cases} \quad (68)$$

- ③ 当第 $n-1$ 节板凳前把手位于调头圆弧 2 时，根据前文情况分析，可得调头圆弧 2 方程（63）与约束圆方程（65）联立得到舞龙队位置求解模型如下：

$$(x_n, y_n) = \arg \min(\theta_n - \theta_{n-1})$$

$$\begin{cases} x = x_{o2} + 2r \cos \theta_{o2} \\ y = y_{o2} + 2r \sin \theta_{o2} \\ x = x_{n-1} + d_{n-1} \cos \theta \\ y = y_{n-1} + d_{n-1} \sin \theta \\ x_{n-1} = b\theta_{n-1} \cos \theta_{n-1} \\ y_{n-1} = b\theta_{n-1} \sin \theta_{n-1} \\ \theta_n - \theta_{n-1} > 0 \\ \theta_{o2 \min} \leq \theta_{o2} \leq \theta_{o2 \max} \end{cases} \quad (69)$$

若上述联立方程无解，则说明第 n 节板凳前把手可能位于调头圆弧 1、盘入螺线段，

此时需依次联立调头圆弧 1、盘入螺线段方程得到舞龙队位置求解模型。由于此时模型与前文情况一致，此处不再赘述。

- ④ 当第 $n-1$ 节板凳前把手位于**盘出螺线段**时，同理将盘出螺线段方程（64）与约束圆方程（65）联立，得到舞龙队位置求解模型如下：

$$(x_n, y_n) = \arg \min(\theta_n - \theta_{n-1})$$

$$\begin{cases} x = -b\theta \cos \theta \\ y = -b\theta \sin \theta \\ x = x_{n-1} + d_{n-1} \cos \theta \\ y = y_{n-1} + d_{n-1} \sin \theta \\ x_{n-1} = -b\theta_{n-1} \cos \theta_{n-1} \\ y_{n-1} = -b\theta_{n-1} \sin \theta_{n-1} \\ \theta_n - \theta_N > 0 \\ \theta_N \leq \theta \leq \theta_0 \end{cases} \quad (70)$$

若上述联立方程无解，则说明第 n 节板凳前把手可能位于调头圆弧 2、调头圆弧 1、盘入螺线段。同理，此时需依次联立调头圆弧 2、调头圆弧 1、盘入螺线段方程得到舞龙队位置求解模型。此处不再赘述。

综上所述，可以求得舞龙队在该范围内的行进过程位置，然后基于前文分析，根据基于方向向量的板凳龙四周定点计算模型和碰撞判断模型得到板凳之间碰撞情况约束。

舞龙队位置和速度的求解

在确定最优调头路径后，舞龙队行进过程的运动轨迹即可确定。本文将在以下篇幅探讨-100s 开始到 100s 为止间 t 时刻板凳龙的位置和速度。根据问题一中的分析，通过速度分解可以求得每个位置的板凳龙的行进速度。而速度分量间的夹角与该点所位于的曲线切向量相关，故在本问多路径组合的速度求解中，需如上文所示将整个行进过程分为盘入螺线段、调头圆弧 1、调头圆弧 2 以及盘出螺线段 4 段，分别求解龙头位于该 4 段曲线时整个舞龙队的位置和速度。

首先，对 t 时刻龙头所处的位置进行判断。以调头开始时间为零时刻，即以龙头开始进入调头圆弧 1 时为零时刻。则前 100s 龙头位于盘入螺线段。可利用前文中基于弧长公式的坐标反解模型进行判断，调头圆弧 1 与调头圆弧 2 段根据上述优化模型求得的调头圆弧弧长与经过行进的弧长进行判断。设 $0-t_1$ 龙头位于调头圆弧 1， t_1-t_2 龙头位于调头圆弧 2， t_2-100s 龙头位于盘出螺线段。具体判断过程如下：

$$\begin{cases} L_i = V_0 \Delta t_i, & i = 1, 2, 3, 4 \\ L_i = b[\frac{1}{2}\theta\sqrt{\theta^2+1} + \frac{1}{2}\ln(\theta + \sqrt{\theta^2+1})]_{\theta_2^1}^{\theta_1}, & i = 1, 4 \\ L_2 = (\theta_{o1\max} - \theta_{o1\min}) \cdot 2r \\ L_3 = (\theta_{2\max} - \theta_{o2\min}) \cdot r \\ \Delta t_1 = 100, & \Delta t_2 = t_1 \\ \Delta t_3 = t_2 - t_1, & \Delta t_4 = 100 - t_2 \\ v_0 = 1 \end{cases} \quad (71)$$

通过上述方程可解得 θ_0 、 t_1 、 t_2 、 θ'_1 ，由此可得到-100s-100s 内 t 时刻龙头所处的位

置以及龙头在盘出螺线上起始点与盘入螺线上的终止点。

然后，根据问题一中对龙头与其余各节板凳龙的位置关系分析，可知利用把手位置模型，通过 t 时刻龙头位置可以求得整个舞龙队 t 时刻的位置。

最后，根据舞龙队速度求解模型分析可知，通过该四段曲线的切向量得到速度分量间夹角关系，从而根据同一时刻前后板凳龙的速度关系（18）对该时刻整个舞龙对的速度进行迭代求解。由于与问题一中求解过程一致，本文于此不再赘述，其问题关键在于板凳龙所处曲线切向量的求解。根据该四段曲线的参数方程，可以求得其切向量如下：

$$\begin{cases} \vec{n}_1 = \left(\frac{dx}{d\theta}, \frac{dy}{d\theta} \right) = (b \cos \theta_{n-1} - b\theta_{n-1} \sin \theta_{n-1}, b \sin \theta_{n-1} + b\theta_{n-1} \cos \theta_{n-1}) \\ \vec{n}_4 = \left(\frac{dx}{d\theta}, \frac{dy}{d\theta} \right) = (b\theta_{n-1} \sin \theta_{n-1} - b \cos \theta_{n-1}, -(b\theta_{n-1} \cos \theta_{n-1} + b \sin \theta_{n-1})) \\ \vec{n}_{o1} = (x_{o1} - \sin \theta_{n-1}, y_{o1} + \cos \theta_{n-1}) \\ \vec{n}_{o2} = (x_{o2} - \sin \theta_{n-1}, y_{o2} + \cos \theta_{n-1}) \end{cases} \quad (72)$$

8.2 问题四的模型求解

8.2.1 算法设计

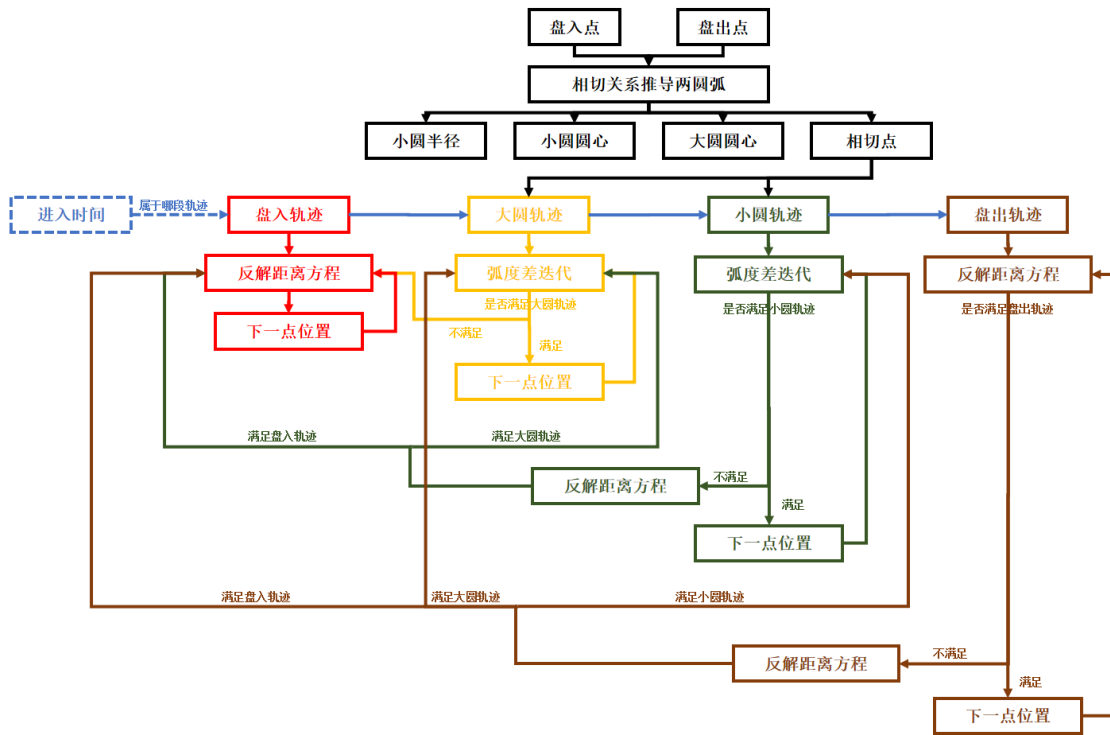


图 14 流程图

Step1: 求解圆弧参数

使用上述圆弧反解公式求解出调头大圆圆心 O_1 ，小圆圆心 O_2 ，两圆相切点 T_0 ，由此可以得出每段圆弧的轨迹。

Step2: 龙头位置判断

根据龙头行动距离判断其所处的曲线位置，从而能进一步迭代出下一把手位置。

Step3: 把手位置判断

对于前一把手位置, 需要动态决定后一把手所处的曲线, 具体来说: 当前一把手处于盘出螺线上时, 则需按顺序判断后一把手是否位于盘出螺线、小圆曲线、大圆曲线、盘入曲线上; 当前一把手处于小圆曲线上时, 则需按顺序判断后一把手是否位于小圆曲线、大圆曲线、盘入曲线上; 当前一把手处于大圆曲线上时, 则需按顺序判断后一把手是否位于大圆曲线、盘入曲线上; 当前一把手处于盘入曲线上时, 则后一把手位于盘入曲线上。

Step4: 把手位置迭代

若前后把手均在大圆曲线或小圆曲线上时, 由于求解中弦长固定, 对应的弧度固定, 则可通过弧度差得到后一把手的极坐标位置以简化计算; 而对于其他情况, 则需求解非线性方程进行求解。

Step5: 碰撞约束

在 $[0.1, 9 * \pi / 1.7]$ 中以步长为 0.1 遍历盘入 (盘出) 角。而对于任意角度中, 最短进入调头曲线的时间为 1330s, 最晚出调头曲线的时间为 1405s, 因此为减少计算量, 仅考虑时间在 $[1330s, 1405s]$ 的碰撞关系。求解后发现当角度大于 6.3 时, 始终满足板凳不碰撞, 在此基础上进行下一步。

Step6: 刚体约束

考虑到在全程中第二个把手应始终向前, 因此仅考虑龙头前把手刚到盘出螺线上时, 其切线与龙头径向角度, 使用粗细网格遍历, 粗遍历为 $[6.4 : 0.1 : 9 * \pi / 1.7]$, 找到龙头前把手刚到盘出螺线上的时刻, 计算余弦值后退出内层遍历, 当余弦值大于 0 时则退出整体循环。之后使用细遍历为 $[15.2 : 0.01 : 15.4]$, 求解得到结果如下表所示:

表 4 位置结果

	-100s	-50s	0s	50s	100s
龙头 x(m)	31.195992	24.572940	15.320000	44.388713	65.471438
龙头 y(m)	8.237178	5.633709	-3.836979	-11.031671	15.528573
第 1 节龙身 x(m)	31.534480	25.002394	16.007217	44.149585	65.309678
第 1 节龙身 y(m)	8.472188	6.707346	-4.138485	-11.778506	13.918570
第 51 节龙身 x(m)	40.060731	35.154154	29.455237	36.594808	60.460295
第 51 节龙身 y(m)	-7.705769	-7.868149	-3.029126	-4.453133	11.743435
第 101 节龙身 x(m)	47.064894	42.966918	38.446300	27.001258	55.186302
第 101 节龙身 y(m)	-12.711884	6.129980	7.631054	2.143023	-3.089670
第 151 节龙身 x(m)	53.153055	49.561598	45.698899	16.182152	49.351700
第 151 节龙身 y(m)	-13.919775	10.222563	-1.796426	-3.895212	-8.155265
第 201 节龙身 x(m)	58.611691	55.375700	51.947557	29.550337	42.727317
第 201 节龙身 y(m)	-7.494408	5.804532	-1.560805	-2.322950	-3.590584
龙尾 (后) x(m)	60.858470	57.748611	54.470141	33.791835	39.461675
龙尾 (后) y(m)	-6.451404	5.662546	-7.165359	-6.591148	2.034944

表 5 速度结果

	-100s	-50s	0s	50s	100s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.999895	0.999727	0.998167	1.000026	1.000005
第 51 节龙身 (m/s)	0.999299	0.998490	0.993822	1.000390	1.000066
第 101 节龙身 (m/s)	0.999031	0.998073	0.993082	1.001367	1.000151
第 151 节龙身 (m/s)	0.998880	0.997864	0.992777	0.940454	1.000279
第 201 节龙身 (m/s)	0.998782	0.997739	0.992609	0.936476	1.000492
龙尾 (后) (m/s)	0.998750	0.997698	0.992558	0.936081	1.000640

表 6 结果分析

	盘入（盘出）角	曲线长度	优化幅度
调整前	16.56	13.62	8.15%
调整后	15.32	12.51	

舞龙队在调整曲线上运动轨迹如下图所示，非常平滑，未出现刚体卡住现象：

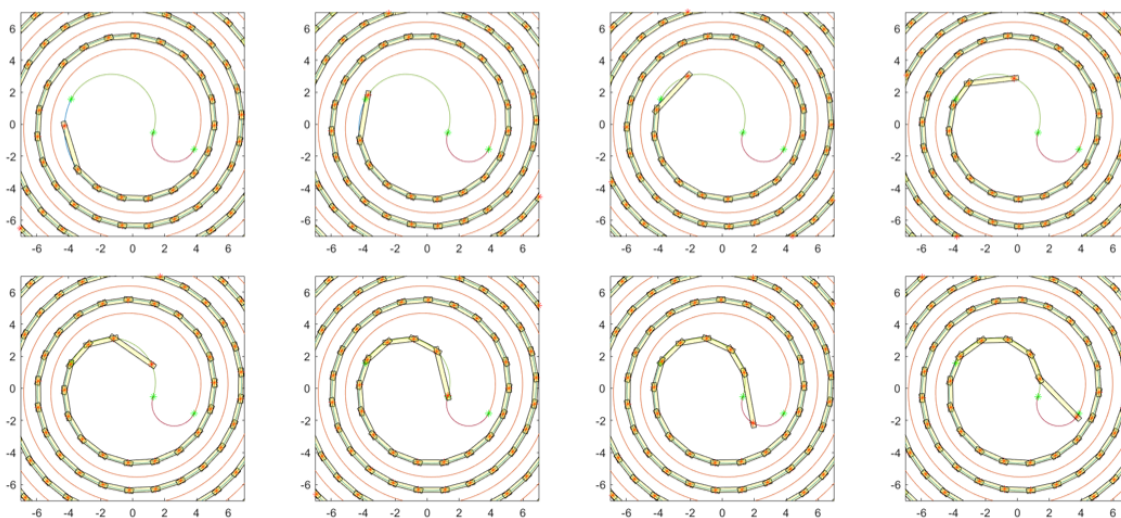


图 15 运动轨迹图

九、第五问模型的建立与求解

9.1 问题五模型的建立

本文要求在舞龙队沿问题四中路径行进，龙头行进速度保持不变的情况下，确定满足舞龙队个把手的速度均不超过 2m/s 约束的最大龙头行进速度。

9.2 最大龙头行进速度模型

决策变量：龙头的行进速度 V_0

本问中舞龙队沿问题四设定的路径行进，且龙头的行进速度不变。根据前文分析可得，已知整个舞龙队位置以及龙头行进速度的条件下，舞龙队各把手的速度均可求得。因此确定龙头的行进速度 V_0 为决策变量。进一步，结合题目背景将其转化为数学形式的约束条件。

约束条件：舞龙队行进过程中各把手速度约束

根据问题四舞龙队位置和速度的求解模型中对 t 时刻龙头所处的位置判断分析，可知在龙头的行进速度为 V_0 情况下，龙头的运动情况可由下述方程求得：

$$\begin{cases} L_i = V_0 \Delta t_i, & i = 1, 2, 3, 4 \\ L_i = b[\frac{1}{2}\theta\sqrt{\theta^2 + 1} + \frac{1}{2}\ln(\theta + \sqrt{\theta^2 + 1})]_{\theta_1}^{\theta_0}, & i = 1, 4 \\ L_2 = (\theta_{o1\max} - \theta_{o1\min}) \cdot 2r \\ L_3 = (\theta_{2\max} - \theta_{o2\min}) \cdot r \\ \Delta t_1 = T_1, \quad \Delta t_2 = t_1 \\ \Delta t_3 = t_2 - t_1, \quad \Delta t_4 = T_2 - t_2 \\ v_0 = 1 \quad \theta_0 = 0, \theta = 16\pi \end{cases} \quad (73)$$

本问仍以调头开始时间，即以龙头开始进入调头圆弧 1 时为零时刻。通过求解上述方程可解得 θ_0 、 T_1 、 T_2 、 θ' ，由此可得到-100s-100s 内 t 时刻龙头所处的位置以及龙头在盘出螺线上起始点与盘入螺线上的终止点。其中，0-t1 龙头位于调头圆弧 1，t1-t2 龙头位于调头圆弧 2，t2-100s 龙头位于盘出螺线段。由问题四中舞龙队位置求解模型四种情况下的探讨，可以得到 t 时刻板凳龙的位置。结合舞龙队速度求解模型，可得到 t 时刻每个舞龙队的速度。

本文要求舞龙队个把手的速度均不超过 2m/s，则有以下约束条件：

$$\begin{cases} v_n(t) < 2 \\ \forall i \in \{0, 1, \dots, 22\} \\ \forall t \in [T_2, T_1] \end{cases} \quad (74)$$

结合上文分析的约束条件，本文的得到了最大龙头行进速度模型如下：

$$\max v_0 \quad \begin{cases} v_n(t) < 2 \\ \forall i \in \{0, 1, \dots, 22\} \\ \forall t \in [T_2, T_1] \end{cases} \quad (75)$$

9.3 结果分析

作出速度最大值变化图如下所示，可以发现在 [-100s,0s] 时，速度均为 1m/s，到了 [0s,72s] 时，由于舞龙队正处在圆弧与螺线交界处，速度出现巨大波动，而到了 [72s,100s] 时，速度大于 1m/s，但正逐渐向 1m/s 靠近，趋于稳定。

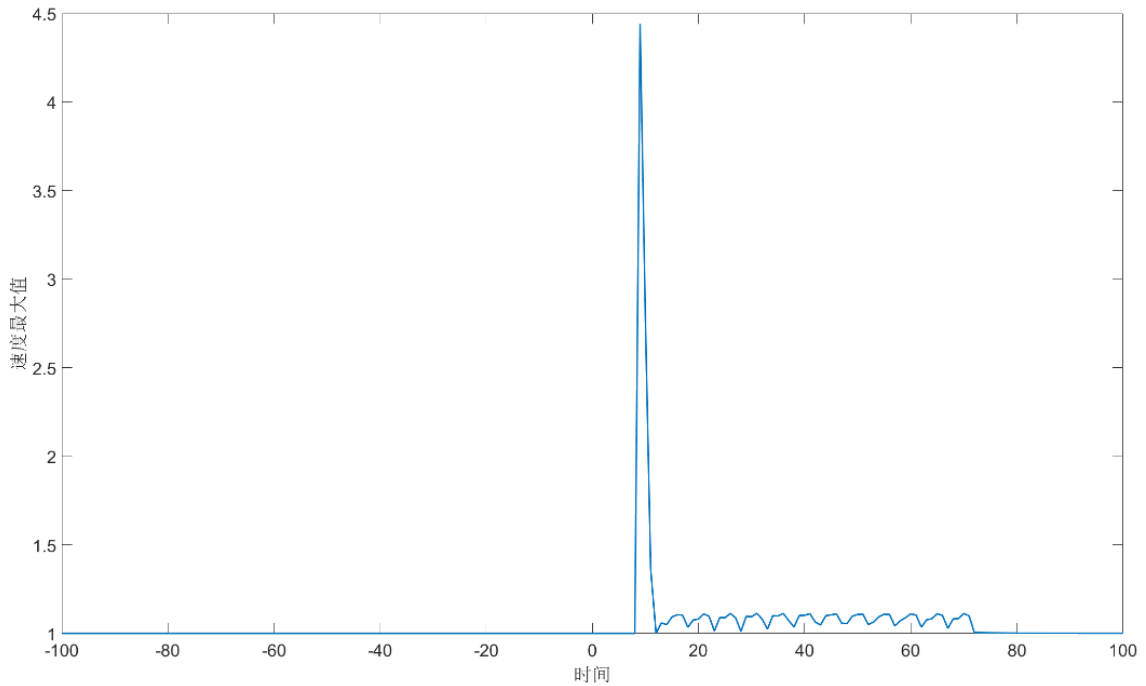


图 16 速度最大值变化图

因此选择时间范围为 $[-100s, 100s]$ 可以找到在时间为 9s 时，全局的速度最大值为 $4.441m/s$ ，为第三个把手，由于问题一可知，对于任意龙头初速度，其他把手与龙头初速度比值始终保持不变，若要使各把手速度均不超过 $2m/s$ ，则根据比值关系可得龙头前进速度为 $0.45m/s$ 。

对于计算得到结果进行验证，得到结果如图所示，可见其中最大为 $2m/s$ ，满足问题条件。

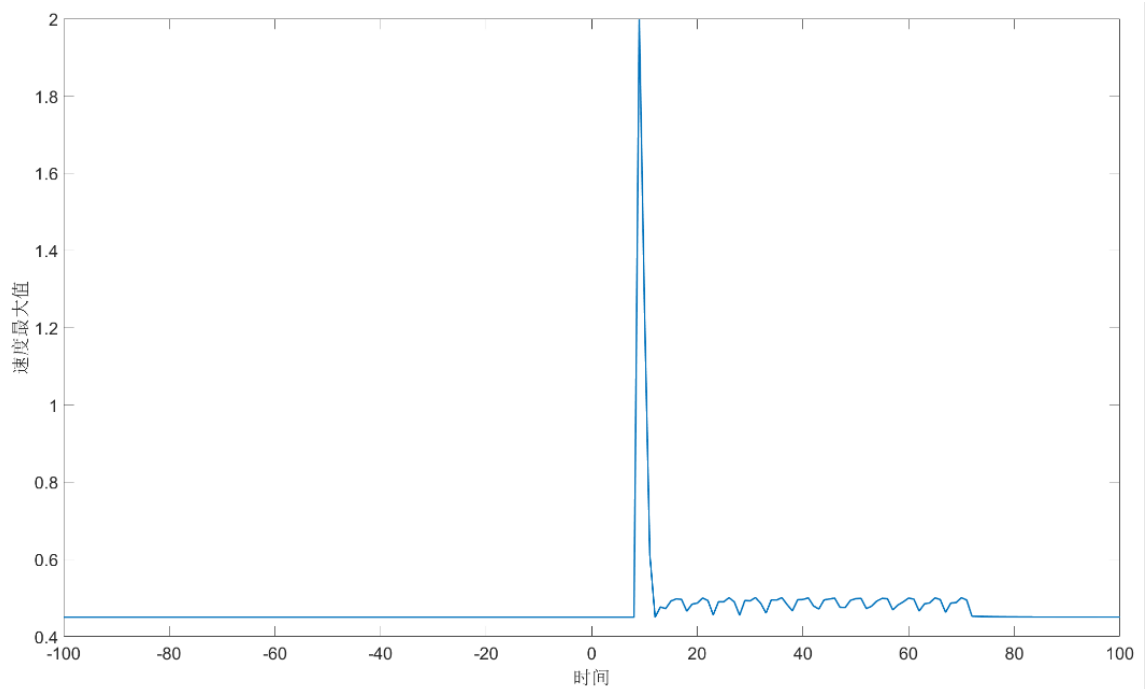


图 17 验证图

十、模型的推广应用

该模型的推广可以应用于多个实际场景，具有广泛的适用性。首先，模型中的路径规划和速度优化思想可用于物流和交通调度中的车辆或机器人路径优化，确保在复杂环境下高效运行。其次，涉及的刚体约束与碰撞检测技术在机械设计、机器人技术及自动驾驶中有着重要应用，能够保障在运动过程中避免碰撞。此外，模型使用的改进牛顿法和粗细网格搜索等优化技术，也可推广到工程设计、资源配置和实验参数优化等多领域，帮助解决复杂的多变量约束问题。最后，这一模型为传统文化活动的分析和优化提供了工具，不仅适用于“板凳龙”表演，还能用于其他传统艺术和竞技项目的动作优化，促进文化传承与现代创新的结合，使传统文化焕发新的生机。

参考文献

- [1] 刘崇军. 等距螺旋的原理与计算. [J]. 北京: 数学的实践与认识, 2018, 48 (11): 165-174
- [2] 韩中庚. 数学建模方法及其应用 [M]. 高等教育出版社,2009.
- [3] 百度百科. 阿基米德螺线 [EB/OL].[.] https://baike.baidu.com/item/%E9%98%BF%E5%9F%BA%E7%B1%B3%E5%BE%B7%E8%9E%BA%E7%BA%BF?fromModule=lemma_search-box
- [3] 张少川. 渐开线及阿基米德螺旋线绘图仪:CN 94214195[P] CN 2204725 Y[2024-09-08].DOI:CN2204725 Y.

```

clear
clc

%% 问题一

% 等螺距为 0.55
b = 0.55 / (2 * pi);
resultx = zeros(448,301);
resultv = zeros(224,301);

for t = 0:300

    %% 计算第一个点的运动

    theta1 = 32 * pi;
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) (0.55/(2*pi)) * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1
+ sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 +
sqrt(theta2^2 + 1))) - t;
    f_prime = @(theta2) - (0.55/(2*pi)) * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2
+ 1));

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
        iteration = iteration + 1;
    end

    %% 计算某个时间下的所有点

    result = zeros(224,1);
    % 第一个点
    theta1 = theta2;
    r = b * theta1;
    x1 = r .* cos(theta1);
    y1 = r .* sin(theta1);
    result(1,1) = theta1;
    resultx(1,t + 1) = x1;

```

```
resultx(2,t + 1) = y1;
```

```
%% 牛顿法求解后续点
```

```
% 第二个点
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
```

```
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```
while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
    theta2 = theta2 - f(theta2) / f_prime(theta2);
```

```
    iteration = iteration + 1;
```

```
end
```

```
% 计算第二个点的坐标
```

```
r2 = b * theta2;
```

```
x2 = r2 * cos(theta2);
```

```
y2 = r2 * sin(theta2);
```

```
x1 = x2;
```

```
y1 = y2;
```

```
theta1 = theta2;
```

```
result(2,1) = theta1;
```

```
resultx(3,t + 1) = x1;
```

```
resultx(4,t + 1) = y1;
```

```
% 第三个及之后的点
```

```
for i = 3:224
```

```
    % 定义目标函数 f 和数值导数 f_prime
```

```
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 1.65;
```

```
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```

iteration = 0;

% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(i,1) = theta1;
resultx(2*i - 1,t + 1) = x1;
resultx(2*i,t + 1) = y1;
end

%% 计算速度
resultv(:,t + 1) = velocity(result, 0.55, 1);
end

resultx = round(resultx, 6);
resultv = round(resultv, 6);

answerx = resultx(:,[1, 61, 121, 181, 241, 301]);
answerx = answerx([1, 2, 3, 4, 103, 104, 203, 204, 303, 304, 403, 404, 447, 448],:);
answerv = resultv(:,[1, 61, 121, 181, 241, 301]);
answerv = answerv([1, 2, 52, 102, 152, 202, 224],:);

% 300s 时各把手速度变化情况
plot(resultv(:,301), 'LineWidth', 1.5)
xlim([1,224])
xlabel('把手序号', 'FontSize', 16)
ylabel('速度变化', 'FontSize', 16)
ax = gca;
ax.FontSize = 16;

% 龙尾速度相对于时间的变化情况
plot(resultv(224,:), 'LineWidth', 1.5)
xlim([1,301])
xlabel('时间', 'FontSize', 16)

```



```
ylabel('速度变化', 'FontSize', 16)
```

```
ax = gca;
```

```
ax.FontSize = 16;
```

```
clear
```

```
clc
```

```
%% 问题一（灵敏度分析 1）
```

```
p = 1;
```

```
for P = 0.2:0.01:2
```

```
    % 等螺距为
```

```
    b = P / (2 * pi);
```

```
%% 计算第一个点的运动
```

```
    theta1 = 32 * pi;
```

```
    % 定义目标函数 f 和数值导数 f_prime
```

```
    f = @(theta2) b * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1 +  
sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 +  
sqrt(theta2^2 + 1)));
```

```
    f_prime = @(theta2) - b * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2 + 1));
```

```
    % 初始猜测值
```

```
    theta2 = theta1;
```

```
    tolerance = 1e-6; % 收敛精度
```

```
    max_iterations = 100; % 最大迭代次数
```

```
    iteration = 0;
```

```
    % 牛顿法迭代
```

```
    while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
        theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
```

```
        iteration = iteration + 1;
```

```
    end
```

```
%% 计算某个时间下的所有点
```

```
    result = zeros(224,1);
```

```
    % 第一个点
```

```
    theta1 = theta2;
```

```
    r = b * theta1;
```

```
    x1 = r .* cos(theta1);
```

```
    y1 = r .* sin(theta1);
```

```
    result(1,1) = theta1;
```

```
%% 牛顿法求解后续点
```

```
% 第二个点
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
```

```
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```
while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
    theta2 = theta2 - f(theta2) / f_prime(theta2);
```

```
    iteration = iteration + 1;
```

```
end
```

```
% 计算第二个点的坐标
```

```
r2 = b * theta2;
```

```
x2 = r2 * cos(theta2);
```

```
y2 = r2 * sin(theta2);
```

```
x1 = x2;
```

```
y1 = y2;
```

```
theta1 = theta2;
```

```
result(2,1) = theta1;
```

```
% 第三个及之后的点
```

```
for i = 3:224
```

```
    % 定义目标函数 f 和数值导数 f_prime
```

```
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 1.65;
```

```
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```

while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(i,1) = theta1;
end

%% 计算速度
resultv(:,p) = velocity(result, P, 1);
p = p + 1;
end

% 龙尾关于螺距变化的速度变化
plot(linspace(0.2, 2, 181), resultv(224,:), 'LineWidth', 1.5)
xlim([0.2,2])
xlabel('螺距变化', 'FontSize', 16)
ylabel('速度变化', 'FontSize', 16)
ax = gca;
ax.FontSize = 16;

clear
clc
%% 问题一（灵敏度分析 1）

% 等螺距为 0.55
b = 0.55 / (2 * pi);

p = 1;
for v0 = 0.1:0.01:2
    %% 计算第一个点的运动

    theta1 = 32 * pi;
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) b * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1 +

```

```

sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 +
sqrt(theta2^2 + 1)));
    f_prime = @(theta2) - b * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2 + 1));

% 初始猜测值
theta2 = theta1;
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end

%% 计算某个时间下的所有点

result = zeros(224,1);
% 第一个点
theta1 = theta2;
r = b * theta1;
x1 = r .* cos(theta1);
y1 = r .* sin(theta1);
result(1,1) = theta1;

%% 牛顿法求解后续点

% 第二个点
% 定义目标函数 f 和数值导数 f_prime
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) -
y1)^2) - 2.86;
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;

% 初始猜测值
theta2 = theta1;
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2);
    iteration = iteration + 1;

```

```

end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(2,1) = theta1;

% 第三个及之后的点
for i = 3:224
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)
- y1)^2) - 1.65;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
        iteration = iteration + 1;
    end

    % 计算第二个点的坐标
    r2 = b * theta2;
    x2 = r2 * cos(theta2);
    y2 = r2 * sin(theta2);
    x1 = x2;
    y1 = y2;
    theta1 = theta2;
    result(i,1) = theta1;
end

%% 计算速度
resultv(:,p) = velocity(result, 0.55, v0);
p = p + 1;
end

```

```

% 龙尾关于于龙头初速度变化情况
plot(linspace(0.1, 2, 191), resultv(224,:), 'LineWidth', 1.5)
xlim([0.1,2])
xlabel('龙头速度变化', 'FontSize', 16)
ylabel('龙尾速度变化', 'FontSize', 16)
ax = gca;
ax.FontSize = 16;

clear

clc

%% 问题二

% 等螺距为 0.55
b = 0.55 / (2 * pi);

for t = 412:0.001:414

    %% 计算第一个点的运动

    theta1 = 32 * pi;
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) (0.55/(2*pi)) * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1
+ sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 +
sqrt(theta2^2 + 1))) - t;
    f_prime = @(theta2) - (0.55/(2*pi)) * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2
+ 1));

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
        iteration = iteration + 1;
    end

    %% 计算某个时间下的所有点

    result = zeros(50,3);

```

```
% 绘制第一个点
```

```
theta1 = theta2;
```

```
r = b * theta1;
```

```
x1 = r .* cos(theta1);
```

```
y1 = r .* sin(theta1);
```

```
result(1,1) = theta1;
```

```
result(1,2) = x1;
```

```
result(1,3) = y1;
```

```
%% 牛顿法求解后续点
```

```
% 第二个点
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
```

```
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```
while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
```

```
    iteration = iteration + 1;
```

```
end
```

```
% 计算第二个点的坐标
```

```
r2 = b * theta2;
```

```
x2 = r2 * cos(theta2);
```

```
y2 = r2 * sin(theta2);
```

```
x1 = x2;
```

```
y1 = y2;
```

```
theta1 = theta2;
```

```
result(2,1) = theta1;
```

```
result(2,2) = x1;
```

```
result(2,3) = y1;
```

```
% 第三个及之后的点
```

```
for i = 3:50
```

```
    % 定义目标函数 f 和数值导数 f_prime
```

```
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)
```

```

- y1)^2) - 1.65;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

% 初始猜测值
theta2 = theta1;
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(i,1) = theta1;
result(i,2) = x1;
result(i,3) = y1;
end

overlap = 0;
%% 判断第一个矩形是否与其他矩形重叠
x1 = result(1,2);
y1 = result(1,3);
x2 = result(1 + 1,2);
y2 = result(1 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices1 = [[x1,y1] - 0.15 * N - 0.275 * L;
    [x1,y1] + 0.15 * N - 0.275 * L;
    [x2,y2] + 0.15 * N + 0.275 * L;
    [x2,y2] - 0.15 * N + 0.275 * L;];
for i = 3:49
    if result(i,1) - result(1,1) < 4*pi
        x1 = result(i,2);
        y1 = result(i,3);
        x2 = result(i + 1,2);

```



```

        y2 = result(i + 1,3);
        L = [x2 - x1, y2 - y1];
        N = [-L(2), L(1)] ./ norm(L);
        L = L ./ norm(L);
        vertices = [[x1,y1] - 0.15 * N - 0.275 * L;
                    [x1,y1] + 0.15 * N - 0.275 * L;
                    [x2,y2] + 0.15 * N + 0.275 * L;
                    [x2,y2] - 0.15 * N + 0.275 * L;];
        overlap = overlap + isOverlap(vertices1,vertices);
    end
end

%% 判断第二个矩形是否与其他矩形重叠
x1 = result(2,2);
y1 = result(2,3);
x2 = result(2 + 1,2);
y2 = result(2 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices1 = [[x1,y1] - 0.15 * N - 0.275 * L;
              [x1,y1] + 0.15 * N - 0.275 * L;
              [x2,y2] + 0.15 * N + 0.275 * L;
              [x2,y2] - 0.15 * N + 0.275 * L;];
for i = 4:49
    if result(i,1) - result(1,1) < 4*pi
        x1 = result(i,2);
        y1 = result(i,3);
        x2 = result(i + 1,2);
        y2 = result(i + 1,3);
        L = [x2 - x1, y2 - y1];
        N = [-L(2), L(1)] ./ norm(L);
        L = L ./ norm(L);
        vertices = [[x1,y1] - 0.15 * N - 0.275 * L;
                    [x1,y1] + 0.15 * N - 0.275 * L;
                    [x2,y2] + 0.15 * N + 0.275 * L;
                    [x2,y2] - 0.15 * N + 0.275 * L;];
        overlap = overlap + isOverlap(vertices1,vertices);
    end
end
if overlap > 0
    break
end
end
end

```

```

clear
clc
%% 问题二（结果）

% 等螺距为 0.55
b = 0.55 / (2 * pi);
resultx = zeros(224,2);
resultv = zeros(224,1);
t = 412.47;

%% 计算第一个点的运动

theta1 = 32 * pi;
% 定义目标函数 f 和数值导数 f_prime
f = @(theta2) (0.55/(2*pi)) * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1 + sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 + sqrt(theta2^2 + 1))) - t;
f_prime = @(theta2) - (0.55/(2*pi)) * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2 + 1));

% 初始猜测值
theta2 = theta1;
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end

%% 计算某个时间下的所有点

result = zeros(224,1);
% 第一个点
theta1 = theta2;
r = b * theta1;
x1 = r .* cos(theta1);
y1 = r .* sin(theta1);
result(1,1) = theta1;
resultx(1,1) = x1;

```

```
resultx(1,2) = y1;
```

```
%% 牛顿法求解后续点
```

```
% 第二个点
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
```

```
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```
while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
    theta2 = theta2 - f(theta2) / f_prime(theta2);
```

```
    iteration = iteration + 1;
```

```
end
```

```
% 计算第二个点的坐标
```

```
r2 = b * theta2;
```

```
x2 = r2 * cos(theta2);
```

```
y2 = r2 * sin(theta2);
```

```
x1 = x2;
```

```
y1 = y2;
```

```
theta1 = theta2;
```

```
result(2,1) = theta1;
```

```
resultx(2,2) = x1;
```

```
resultx(2,2) = y1;
```

```
% 第三个及之后的点
```

```
for i = 3:224
```

```
    % 定义目标函数 f 和数值导数 f_prime
```

```
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 1.65;
```

```
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
    % 初始猜测值
```

```
    theta2 = theta1;
```

```
    tolerance = 1e-6; % 收敛精度
```

```
    max_iterations = 100; % 最大迭代次数
```

```

iteration = 0;

% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(i,1) = theta1;
resultx(i, 1) = x1;
resultx(i, 2) = y1;
end

%% 计算速度
resultv(:,1) = velocity(result, 0.55, 1);

resultx = round(resultx, 6);
resultv = round(resultv, 6);

answerx = resultx([1, 2, 52, 102, 152, 202, 224],:);
answerv = resultv([1, 2, 52, 102, 152, 202, 224],:);

clear
clc
%% 问题二（灵敏度分析）

% 等螺距为 0.55
b = 0.55 / (2 * pi);

height = 0.275;

p = 1;
for width = 0.1:0.01:0.3
    for t = 0:0.1:440

```

```
%% 计算第一个点的运动
```

```
theta1 = 32 * pi;
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) (0.55/(2*pi)) * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 *  
log(theta1 + sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2  
+ sqrt(theta2^2 + 1))) - t;
```

```
f_prime = @(theta2) - (0.55/(2*pi)) * (theta2 / sqrt(theta2^2 + 1) +  
sqrt(theta2^2 + 1));
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```
while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
```

```
    iteration = iteration + 1;
```

```
end
```

```
%% 计算某个时间下的所有点
```

```
result = zeros(50,3);
```

```
% 绘制第一个点
```

```
theta1 = theta2;
```

```
r = b * theta1;
```

```
x1 = r .* cos(theta1);
```

```
y1 = r .* sin(theta1);
```

```
result(1,1) = theta1;
```

```
result(1,2) = x1;
```

```
result(1,3) = y1;
```

```
%% 牛顿法求解后续点
```

```
% 第二个点
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)  
- y1)^2) - 2.86;
```

```
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;
```

```
% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end
```

```
% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(2,1) = theta1;
result(2,2) = x1;
result(2,3) = y1;
```

```
% 第三个及之后的点
```

```
for i = 3:50
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 *
sin(theta2) - y1)^2) - 1.65;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
theta2 = theta1;
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;
```

```
% 牛顿法迭代
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
    iteration = iteration + 1;
end
```

```
% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
```

```

x1 = x2;
y1 = y2;
theta1 = theta2;
result(i,1) = theta1;
result(i,2) = x1;
result(i,3) = y1;
end
overlap = 0;
%% 判断第一个矩形是否与其他矩形重叠
x1 = result(1,2);
y1 = result(1,3);
x2 = result(1 + 1,2);
y2 = result(1 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices1 = [[x1,y1] - width * N - height * L;
[x1,y1] + width * N - height * L;
[x2,y2] + width * N + height * L;
[x2,y2] - width * N + height * L;];
for i = 3:49
    if result(i,1) - result(1,1) < 4*pi
        x1 = result(i,2);
        y1 = result(i,3);
        x2 = result(i + 1,2);
        y2 = result(i + 1,3);
        L = [x2 - x1, y2 - y1];
        N = [-L(2), L(1)] ./ norm(L);
        L = L ./ norm(L);
        vertices = [[x1,y1] - width * N - height * L;
[x1,y1] + width * N - height * L;
[x2,y2] + width * N + height * L;
[x2,y2] - width * N + height * L;];
        overlap = overlap + isOverlap(vertices1,vertices);
    end
end
end

```

```

%% 判断第二个矩形是否与其他矩形重叠
x1 = result(2,2);
y1 = result(2,3);
x2 = result(2 + 1,2);
y2 = result(2 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);

```

```

        L = L ./ norm(L);
        vertices1 = [[x1,y1] - width * N - height * L;
                     [x1,y1] + width * N - height * L;
                     [x2,y2] + width * N + height * L;
                     [x2,y2] - width * N + height * L;];
        for i = 4:49
            if result(i,1) - result(1,1) < 4*pi
                x1 = result(i,2);
                y1 = result(i,3);
                x2 = result(i + 1,2);
                y2 = result(i + 1,3);
                L = [x2 - x1, y2 - y1];
                N = [-L(2), L(1)] ./ norm(L);
                L = L ./ norm(L);
                vertices = [[x1,y1] - width * N - height * L;
                           [x1,y1] + width * N - height * L;
                           [x2,y2] + width * N + height * L;
                           [x2,y2] - width * N + height * L;];
                overlap = overlap + isOverlap(vertices1,vertices);
            end
        end
        if overlap > 0
            break
        end
    end
    T(1,p) = t;
    p = p + 1;
end

% 终止时刻对于板长变化情况
plot(linspace(0.1, 0.3, 21), T(1,:), 'LineWidth', 1.5)
xlim([0.1,0.3])
xlabel('板宽变化', 'FontSize', 16)
ylabel('终止时刻', 'FontSize', 16)
ax = gca;
ax.FontSize = 16;

clear
clc
%% 问题三

for P = 0.42:-0.00001:0.4

```



```
% 等螺距为 P
```

```
b = P / (2 * pi);
```

```
%% 计算某个时间下的所有点
```

```
result = zeros(224,3);
```

```
% 第一个点
```

```
theta1 = 4.5 / b;
```

```
r = b * theta1;
```

```
x1 = r .* cos(theta1);
```

```
y1 = r .* sin(theta1);
```

```
result(1,1) = theta1;
```

```
result(1,2) = x1;
```

```
result(1,3) = y1;
```

```
%% 牛顿法求解后续点
```

```
% 第二个点
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
```

```
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
theta2 = theta1;
```

```
tolerance = 1e-6; % 收敛精度
```

```
max_iterations = 100; % 最大迭代次数
```

```
iteration = 0;
```

```
% 牛顿法迭代
```

```
while abs(f(theta2)) > tolerance && iteration < max_iterations
```

```
    theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
```

```
    iteration = iteration + 1;
```

```
end
```

```
% 计算第二个点的坐标
```

```
r2 = b * theta2;
```

```
x2 = r2 * cos(theta2);
```

```
y2 = r2 * sin(theta2);
```

```
x1 = x2;
```

```
y1 = y2;
```

```
theta1 = theta2;
```

```
result(2,1) = theta1;
```

```
result(2,2) = x1;
```

```

result(2,3) = y1;

% 第三个及之后的点
for i = 3:50
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)
- y1)^2) - 1.65;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - f(theta2) / f_prime(theta2); % 更新 theta2
        iteration = iteration + 1;
    end

    % 计算第二个点的坐标
    r2 = b * theta2;
    x2 = r2 * cos(theta2);
    y2 = r2 * sin(theta2);
    x1 = x2;
    y1 = y2;
    theta1 = theta2;
    result(i,1) = theta1;
    result(i,2) = x1;
    result(i,3) = y1;
end

overlap = 0;
%% 判断第一个矩形是否与其他矩形重叠
x1 = result(1,2);
y1 = result(1,3);
x2 = result(1 + 1,2);
y2 = result(1 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices1 = [[x1,y1] - 0.15 * N - 0.275 * L;
[x1,y1] + 0.15 * N - 0.275 * L;

```

```

        [x2,y2] + 0.15 * N + 0.275 * L;
        [x2,y2] - 0.15 * N + 0.275 * L;];
    for i = 3:49
        if result(i,1) - result(1,1) < 4*pi
            x1 = result(i,2);
            y1 = result(i,3);
            x2 = result(i + 1,2);
            y2 = result(i + 1,3);
            L = [x2 - x1, y2 - y1];
            N = [-L(2), L(1)] ./ norm(L);
            L = L ./ norm(L);
            vertices = [[x1,y1] - 0.15 * N - 0.275 * L;
                [x1,y1] + 0.15 * N - 0.275 * L;
                [x2,y2] + 0.15 * N + 0.275 * L;
                [x2,y2] - 0.15 * N + 0.275 * L;];
            overlap = overlap + isOverlap(vertices1,vertices);
        end
    end

    %% 判断第二个矩形是否与其他矩形重叠
    x1 = result(2,2);
    y1 = result(2,3);
    x2 = result(2 + 1,2);
    y2 = result(2 + 1,3);
    L = [x2 - x1, y2 - y1];
    N = [-L(2), L(1)] ./ norm(L);
    L = L ./ norm(L);
    vertices1 = [[x1,y1] - 0.15 * N - 0.275 * L;
        [x1,y1] + 0.15 * N - 0.275 * L;
        [x2,y2] + 0.15 * N + 0.275 * L;
        [x2,y2] - 0.15 * N + 0.275 * L;];
    for i = 4:49
        if result(i,1) - result(1,1) < 4*pi
            x1 = result(i,2);
            y1 = result(i,3);
            x2 = result(i + 1,2);
            y2 = result(i + 1,3);
            L = [x2 - x1, y2 - y1];
            N = [-L(2), L(1)] ./ norm(L);
            L = L ./ norm(L);
            vertices = [[x1,y1] - 0.15 * N - 0.275 * L;
                [x1,y1] + 0.15 * N - 0.275 * L;
                [x2,y2] + 0.15 * N + 0.275 * L;
                [x2,y2] - 0.15 * N + 0.275 * L;];

```

```

        overlap = overlap + isOverlap(vertices1,vertices);
    end
end
if overlap > 0
    break
end
end

```

```
clear
```

```
clc
```

```
%% 问题四（第一步）
```

```
i = 1;
```

```
for angle = 0.1:0.1:9*pi/1.7
```

```
    for t = 1330:0.1:1405
```

```
        [overlap, L, ~] = HYT(t, angle, angle);
```

```
        if overlap ~= 0
```

```
            L = 0;
```

```
            break
```

```
        end
```

```
    end
```

```
    Length(1, i) = L;
```

```
    i = i + 1;
```

```
end
```

```
toc
```

```
clear
```

```
clc
```

```
%% 问题四（第二步）
```

```
for angle = 15.2:0.01:15.4
```

```
    %% 求解圆的属性
```

```
    [R, center1, center2, tangency] = radius(angle, angle);
```

```
    d = -1;
```

```
    for t = 1330:0.01:1405
```

```
        [~,~,result] = HYT(t, angle, angle);
```

```
        if result(1,4) == 4
```

```
            T = tangency(result(1,:), center1, center2);
```

```
            L = [result(1,2) - result(2,2), result(1,3) - result(2,3)];
```

```
            d = dot(T, L);
```

```
            break
```

```

        end
    end
    if d > 0
        break
    end
end

clear
clc

%% 问题四（结果）
b = 1.7 / (2 * pi);
angle1 = 6;
angle2 = 6;

%% 求解圆的属性
[R, center1, center2, tangency] = radius(angle1, angle2);

%% 第一个圆
r = b * angle1;
x1 = r .* cos(angle1);
y1 = r .* sin(angle1);

% 轨迹的角度范围（按顺时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center1(2), x1 - center1(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = - Theta1;
else
    Theta1 = 2*pi - Theta1;
end
Theta2 = atan2(tangency(2) - center1(2), tangency(1) - center1(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = - Theta2;
else
    Theta2 = 2*pi - Theta2;
end

% 绘制圆（顺时针）
if Theta2 < Theta1
    Theta2 = Theta2 + 2*pi;
end

```

```

Theta_one1 = Theta1;
Theta_one2 = Theta2;

%% 第二个圆
r = -b * angle2;
x1 = r .* cos(angle2);
y1 = r .* sin(angle2);

% 轨迹的角度范围（按逆时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center2(2), x1 - center2(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = 2*pi + Theta1;
end
Theta2 = atan2(tangency(2) - center2(2), tangency(1) - center2(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = 2*pi + Theta2;
end

% 绘制圆（逆时针）
if Theta1 < Theta2
    Theta1 = Theta1 + 2*pi;
end
Theta_two1 = Theta1;
Theta_two2 = Theta2;

%% 起点到盘入点的长度
theta2 = 32 * pi;
L1 = b * (0.5 * theta2 * sqrt(1 + theta2^2) + 0.5 * log(theta2 + sqrt(1 + theta2^2))) -
b * (0.5 * angle1 * sqrt(1 + angle1^2) + 0.5 * log(angle1 + sqrt(1 + angle1^2)));

%% 盘入点到切点的长度
L2 = (Theta_one2 - Theta_one1) * 2 * R;

%% 切点到盘出点的长度
L3 = (Theta_two1 - Theta_two2) * R;

resultx = zeros(448,201);
resultv = zeros(224,201);
i = 1;
for t = L1 - 100:L1 + 100
    [~,~,result] = HYT(t, angle1, angle2);
    resultx(1:2:end, i) = result(:, 1);
    resultx(2:2:end, i) = result(:, 2);

```

```

        resultv(:,i) = new_velocity(result, center1, center2);
    i = i + 1;
end

answerx = resultx(:,[1, 51, 101, 151, 201]);
answerx = answerx([1, 2, 3, 4, 103, 104, 203, 204, 303, 304, 403, 404, 447, 448],:);
answerv = resultv(:,[1, 51, 101, 151, 201]);
answerv = answerv([1, 2, 52, 102, 152, 202, 224],:);

clear
clc

%% 问题五
b = 1.7 / (2 * pi);
angle1 = 15.32;
angle2 = 15.32;

%% 求解圆的属性
[R, center1, center2, tangency] = radius(angle1, angle2);

%% 第一个圆
r = b * angle1;
x1 = r .* cos(angle1);
y1 = r .* sin(angle1);

% 轨迹的角度范围（按顺时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center1(2), x1 - center1(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = - Theta1;
else
    Theta1 = 2*pi - Theta1;
end
Theta2 = atan2(tangency(2) - center1(2), tangency(1) - center1(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = - Theta2;
else
    Theta2 = 2*pi - Theta2;
end

% 绘制圆（顺时针）
if Theta2 < Theta1

```

```

        Theta2 = Theta2 + 2*pi;
    end
    Theta_one1 = Theta1;
    Theta_one2 = Theta2;

%% 第二个圆
    r = -b * angle2;
    x1 = r .* cos(angle2);
    y1 = r .* sin(angle2);

% 轨迹的角度范围（按逆时针切换到值域为[0,2pi]）
    Theta1 = atan2(y1 - center2(2), x1 - center2(1));% 螺圆切点对应角度
    if Theta1 < 0
        Theta1 = 2*pi + Theta1;
    end
    Theta2 = atan2(tangency(2) - center2(2), tangency(1) - center2(1));% 圆圆切点对应角度
    if Theta2 < 0
        Theta2 = 2*pi + Theta2;
    end

% 绘制圆（逆时针）
    if Theta1 < Theta2
        Theta1 = Theta1 + 2*pi;
    end
    Theta_two1 = Theta1;
    Theta_two2 = Theta2;

%% 起点到盘入点的长度
    theta2 = 32 * pi;
    L1 = b * (0.5 * theta2 * sqrt(1 + theta2^2) + 0.5 * log(theta2 + sqrt(1 + theta2^2))) -
        b * (0.5 * angle1 * sqrt(1 + angle1^2) + 0.5 * log(angle1 + sqrt(1 + angle1^2)));

%% 盘入点到切点的长度
    L2 = (Theta_one2 - Theta_one1) * 2 * R;

%% 切点到盘出点的长度
    L3 = (Theta_two1 - Theta_two2) * R;

resultx = zeros(448,201);
resultv = zeros(224,201);
i = 1;
for t = L1 - 100:L1 + 100
    [~,~,result] = HYT(t, angle1, angle2);

```



```

resultx(1:2:end, i) = result(:, 1);
resultx(2:2:end, i) = result(:, 2);
resultv(:,i) = new_velocity(result, center1, center2);
i = i + 1;
end

% 每秒速度最大值
plot(linspace(-100,100,201),max(resultv), 'LineWidth', 1.5)
xlim([-100,100])
xlabel('时间', 'FontSize', 16)
ylabel('速度最大值', 'FontSize', 16)
ax = gca;
ax.FontSize = 16;

clear
clc

%% 问题五（结果展示）
b = 1.7 / (2 * pi);
angle1 = 15.32;
angle2 = 15.32;

%% 求解圆的属性
[R, center1, center2, tangency] = radius(angle1, angle2);

%% 第一个圆
r = b * angle1;
x1 = r .* cos(angle1);
y1 = r .* sin(angle1);

% 轨迹的角度范围（按顺时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center1(2), x1 - center1(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = - Theta1;
else
    Theta1 = 2*pi - Theta1;
end
Theta2 = atan2(tangency(2) - center1(2), tangency(1) - center1(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = - Theta2;
else

```

```

    Theta2 = 2*pi - Theta2;
end

% 绘制圆（顺时针）
if Theta2 < Theta1
    Theta2 = Theta2 + 2*pi;
end
Theta_one1 = Theta1;
Theta_one2 = Theta2;

%% 第二个圆
r = -b * angle2;
x1 = r .* cos(angle2);
y1 = r .* sin(angle2);

% 轨迹的角度范围（按逆时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center2(2), x1 - center2(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = 2*pi + Theta1;
end
Theta2 = atan2(tangency(2) - center2(2), tangency(1) - center2(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = 2*pi + Theta2;
end

% 绘制圆（逆时针）
if Theta1 < Theta2
    Theta1 = Theta1 + 2*pi;
end
Theta_two1 = Theta1;
Theta_two2 = Theta2;

%% 起点到盘入点的长度
theta2 = 32 * pi;
L1 = b * (0.5 * theta2 * sqrt(1 + theta2^2) + 0.5 * log(theta2 + sqrt(1 + theta2^2))) -
b * (0.5 * angle1 * sqrt(1 + angle1^2) + 0.5 * log(angle1 + sqrt(1 + angle1^2)));

%% 盘入点到切点的长度
L2 = (Theta_one2 - Theta_one1) * 2 * R;

%% 切点到盘出点的长度
L3 = (Theta_two1 - Theta_two2) * R;

```

```

resultx = zeros(448,201);
resultv = zeros(224,201);
i = 1;
for t = L1 - 100:L1 + 100
    [~,~,result] = HYT(t, angle1, angle2);
    resultx(1:2:end, i) = result(:, 1);
    resultx(2:2:end, i) = result(:, 2);
    resultv(:,i) = new_velocity(result, center1, center2);
    i = i + 1;
end

% 每秒速度最大值
plot(linspace(-100,100,201),max(resultv), 'LineWidth', 1.5)
xlim([-100,100])
xlabel('时间', 'FontSize', 16)
ylabel('速度最大值', 'FontSize', 16)
ax = gca;
ax.FontSize = 16;

```

```

function resultv = velocity(result, P, v0)
v1 = v0;
resultv = zeros(224,1);
for i = 1:223

    % 初始化参数
    theta1 = result(i,1);
    theta2 = result(i + 1,1);

    % 定义第一个点极坐标到直角坐标的转换
    r1 = P/(2*pi) * theta1;
    x1 = r1 * cos(theta1);
    y1 = r1 * sin(theta1);
    % 计算切线方向向量 T1
    dx_dtheta1 = cos(theta1) - theta1 * sin(theta1);
    dy_dtheta1 = sin(theta1) + theta1 * cos(theta1);
    T1 = [dx_dtheta1, dy_dtheta1]; % 切线方向向量

    % 定义第二个点极坐标到直角坐标的转换
    r2 = P/(2*pi) * theta2;
    x2 = r2 * cos(theta2);
    y2 = r2 * sin(theta2);
    % 计算切线方向向量 T2

```

```

dx_dtheta2 = cos(theta2) - theta2 * sin(theta2);
dy_dtheta2 = sin(theta2) + theta2 * cos(theta2);
T2 = [dx_dtheta2, dy_dtheta2]; % 切线方向向量

% 计算两点之间的连线方向向量 L
L = [x2 - x1, y2 - y1];

% 计算第一个点切线与连线的夹角
c1 = abs(dot(T1, L)/(norm(L)*norm(T1)));
% 计算第二个点切线与连线的夹角
c2 = abs(dot(T2, L)/(norm(L)*norm(T2)));

% 计算并记录
resultv(i,1) = v1;
v2 = v1*c1/c2;
v1 = v2;
end
resultv(end,1) = v1;

function T = tangent(result, center1, center2)
% 在盘入螺线上
if result(1,4) == 1
    theta = result(1,1);
    dx_dtheta = cos(theta) - theta * sin(theta);
    dy_dtheta = sin(theta) + theta * cos(theta);
    T = [dx_dtheta, dy_dtheta];
end

% 在盘入圆弧上
if result(1,4) == 2
    T = [center1(2) - result(1,3), result(1,2) - center1(1)];
end

% 在盘出圆弧上
if result(1,4) == 3
    T = [center2(2) - result(1,3), result(1,2) - center2(1)];
end

% 在盘出螺线上
if result(1,4) == 4
    theta = result(1,1);
    dx_dtheta = cos(theta) - theta * sin(theta);

```

```

dy_dtheta = sin(theta) + theta * cos(theta);
T = - [dx_dtheta, dy_dtheta];
end

```

```

function [R, center1, center2, tangency] = radius(theta1, theta2)

```

```

b = 1.7 / (2 * pi);

```

```

% 顺时针的径向向量与坐标

```

```

dx_dtheta1 = cos(theta1) - theta1 * sin(theta1);

```

```

dy_dtheta1 = sin(theta1) + theta1 * cos(theta1);

```

```

N1 = [-dy_dtheta1, dx_dtheta1] / sqrt(dy_dtheta1^2 + dx_dtheta1^2);

```

```

N1x = N1(1,1);

```

```

N1y = N1(1,2);

```

```

x1 = b * theta1 * cos(theta1);

```

```

y1 = b * theta1 * sin(theta1);

```

```

% 逆时针的径向向量与坐标

```

```

dx_dtheta2 = cos(theta2) - theta2 * sin(theta2);

```

```

dy_dtheta2 = sin(theta2) + theta2 * cos(theta2);

```

```

N2 = [dy_dtheta2, -dx_dtheta2] / sqrt(dy_dtheta2^2 + dx_dtheta2^2);

```

```

N2x = N2(1,1);

```

```

N2y = N2(1,2);

```

```

x2 = - b * theta2 * cos(theta2);

```

```

y2 = - b * theta2 * sin(theta2);

```

```

A = (2*N1x - N2x)^2 + (2*N1y - N2y)^2 - 9;

```

```

B = 2 * (2*N1x - N2x) * (x1 - x2) + 2 * (2*N1y - N2y) * (y1 - y2);

```

```

C = (x1 - x2)^2 + (y1 - y2)^2;

```

```

delta = B^2 - 4 * A * C;

```

```

if abs(A) < 1e-5

```

```

    R = - C / B;

```

```

else

```

```

    R = (-B - sqrt(delta))/(2*A);

```

```

    if R < 0

```

```

        R = (-B + sqrt(delta))/(2*A);

```

```

    end

```

```

end

```

```

center1 = [x1, y1] + N1 .* 2 * R;

```

```

center2 = [x2, y2] + N2 .* R;

```

```

tangency = center1 + (2/3) * (center2 - center1);

```

```
function resultv = new_velocity(result, center1, center2)
```

```
v1 = 1;
```

```
resultv = zeros(224,1);
```

```
for i = 1:223
```

```
    T1 = tangent(result(i,:), center1, center2);
```

```
    T2 = tangent(result(i+1,:), center1, center2);
```

```
    % 计算两点之间的连线方向向量 L
```

```
    L = [result(i+1,2) - result(i,2), result(i+1,3) - result(i,3)];
```

```
    % 计算第一个点切线与连线的夹角
```

```
    c1 = abs(dot(T1, L)/(norm(L)*norm(T1)));
```

```
    % 计算第二个点切线与连线的夹角
```

```
    c2 = abs(dot(T2, L)/(norm(L)*norm(T2)));
```

```
    % 计算并记录
```

```
    resultv(i,1) = v1;
```

```
    v2 = v1*c1/c2;
```

```
    v1 = v2;
```

```
end
```

```
resultv(end,1) = v1;
```

```
function overlap = isOverlap(P1, P2)
```

```
    % 获取矩形 P1 和 P2 的两条边向量
```

```
    edges(1, :) = P1(2, :) - P1(1, :);
```

```
    edges(2, :) = P1(3, :) - P1(2, :);
```

```
    edges(3, :) = P2(2, :) - P2(1, :);
```

```
    edges(4, :) = P2(3, :) - P2(2, :);
```

```
    overlap = 1; % 假设有重叠，直到找到分离轴
```

```
    for i = 1:4
```

```
        % 计算每条边的法向量（垂直于边）
```

```
        edge = edges(i, :);
```

```
        axis = [-edge(2), edge(1)]; % 计算法向量
```

```
        % 将 P1 的顶点投影到当前轴上
```

```
        proj1 = zeros(4, 1);
```

```
        for j = 1:4
```

```
            proj1(j) = dot(P1(j, :), axis) / norm(axis);
```

```
        end
```

```
        p1_min = min(proj1);
```

```

        p1_max = max(proj1);

        % 将 P2 的顶点投影到当前轴上
        proj2 = zeros(4, 1);
        for j = 1:4
            proj2(j) = dot(P2(j, :), axis) / norm(axis);
        end
        p2_min = min(proj2);
        p2_max = max(proj2);

        % 检查投影是否重叠
        if p1_max < p2_min || p2_max < p1_min
            overlap = 0; % 如果有分离轴，返回没有重叠
            return;
        end
    end
end
end

```

```

function [overlap, L, result] = HYT(t, angle1, angle2)
% 绘制螺线
b = 1.7 / (2 * pi);

%% 求解圆的属性
[R, center1, center2, tangency] = radius(angle1, angle2);

%% 绘制第一个圆
r = b * angle1;
x1 = r .* cos(angle1);
y1 = r .* sin(angle1);

% 轨迹的角度范围（按顺时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center1(2), x1 - center1(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = - Theta1;
else
    Theta1 = 2*pi - Theta1;
end
Theta2 = atan2(tangency(2) - center1(2), tangency(1) - center1(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = - Theta2;
else

```

```

    Theta2 = 2*pi - Theta2;
end

% 绘制圆（顺时针）
if Theta2 < Theta1
    Theta2 = Theta2 + 2*pi;
end
Theta_one1 = Theta1;
Theta_one2 = Theta2;

%% 绘制第二个圆
r = -b * angle2;
x1 = r .* cos(angle2);
y1 = r .* sin(angle2);

% 轨迹的角度范围（按逆时针切换到值域为[0,2pi]）
Theta1 = atan2(y1 - center2(2), x1 - center2(1));% 螺圆切点对应角度
if Theta1 < 0
    Theta1 = 2*pi + Theta1;
end
Theta2 = atan2(tangency(2) - center2(2), tangency(1) - center2(1));% 圆圆切点对应角度
if Theta2 < 0
    Theta2 = 2*pi + Theta2;
end

% 绘制圆（逆时针）
if Theta1 < Theta2
    Theta1 = Theta1 + 2*pi;
end
Theta_two1 = Theta1;
Theta_two2 = Theta2;

%% 起点到盘入点的长度
theta2 = 32 * pi;
L1 = b * (0.5 * theta2 * sqrt(1 + theta2^2) + 0.5 * log(theta2 + sqrt(1 + theta2^2))) -
b * (0.5 * angle1 * sqrt(1 + angle1^2) + 0.5 * log(angle1 + sqrt(1 + angle1^2)));

%% 盘入点到切点的长度
L2 = (Theta_one2 - Theta_one1) * 2 * R;

%% 切点到盘出点的长度
L3 = (Theta_two1 - Theta_two2) * R;

```



```

%% 第一段
if t <= L1
    % 计算第一个点的运动（盘入螺旋线）

    theta1 = 32 * pi;
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) (1.7/(2*pi)) * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1 + sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 + sqrt(theta2^2 + 1))) - t;
    f_prime = @(theta2) - (1.7/(2*pi)) * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2 + 1));

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end

    % 绘制第一个点
    theta1 = theta2;
    r = b * theta1;
    x1 = r .* cos(theta1);
    y1 = r .* sin(theta1);
    result(1,1) = theta1;
    result(1,2) = x1;
    result(1,3) = y1;
    result(1,4) = 1;

    % 第二个点
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度

```

```

max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代
theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
    iteration = iteration + 1;
end

% 计算第二个点的坐标
theta1 = theta2;
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
result(2,1) = theta1;
result(2,2) = x1;
result(2,3) = y1;
result(2,4) = 1;

% 第三个及之后的点
for i = 3:224
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)
- y1)^2) - 1.65;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;

    % 初始猜测值
    theta2 = result(i - 1,1);
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end

    % 计算之后点的坐标
    r2 = b * theta2;

```

```

        x2 = r2 * cos(theta2);
        y2 = r2 * sin(theta2);
        x1 = x2;
        y1 = y2;
        theta1 = theta2;
        result(i,1) = theta1;
        result(i,2) = x1;
        result(i,3) = y1;
        result(i,4) = 1;
    end
end

%% 第二段
if L1 < t && t <= L1 + L2

    % 计算第一个点的运动
    theta = (t - L1) / (2 * R);
    x1 = 2*R * cos(theta + Theta_one1) + center1(1);
    y1 = - 2*R * sin(theta + Theta_one1) + center1(2);
    result(1,1) = theta + Theta_one1;
    result(1,2) = x1;
    result(1,3) = y1;
    result(1,4) = 2;

    % 第二个点
    % 首先判断是否在盘入圆弧上
    hyt2 = 0;
    a1 = real(2 * asin(2.86/(2*R)));
    if result(1,1) - a1 > Theta_one1
        x2 = 2*R * cos(result(1,1) - a1) + center1(1);
        y2 = - 2*R * sin(result(1,1) - a1) + center1(2);
        x1 = x2;
        y1 = y2;
        result(2,1) = result(1,1) - a1;
        result(2,2) = x1;
        result(2,3) = y1;
        result(2,4) = 2;
    else
        % 点就在盘入螺旋线上
        % 定义目标函数 f 和数值导数 f_prime
        hyt2 = 1;
        f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)
- y1)^2) - 2.86;

```

```

f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

% 初始猜测值
theta2 = angle1;
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代（防止角度小于盘入点）
k = 1;
while true
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end
    if theta2 > angle1
        break
    else
        theta2 = 1.1^k * angle1;
        k = k + 1;
    end
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(2,1) = theta1;
result(2,2) = x1;
result(2,3) = y1;
result(2,4) = 1;
end

% 第三个及之后的点
for i = 3:224
    a1 = real(2 * asin(1.65/2/(2*R)));
    % 首先判断是否在盘入圆弧上
    if hyt2 ~= 1 && result(i - 1,1) - a1 > Theta_one1 && 4*R > 1.65
        x2 = 2*R * cos(result(i - 1,1) - a1) + center1(1);
        y2 = - 2*R * sin(result(i - 1,1) - a1) + center1(2);
    end
end

```

```

        x1 = x2;
        y1 = y2;
        result(i,1) = result(i - 1,1) - a1;
        result(i,2) = x1;
        result(i,3) = y1;
        result(i,4) = 2;

    else
        % 点就在盘入螺旋线上
        % 定义目标函数 f 和数值导数 f_prime

        f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 *
sin(theta2) - y1)^2) - 1.65;
        f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

        % 初始猜测值
        % 上一次是在圆弧上还是螺旋线上
        if hyt2 ~= 0
            theta2 = result(i - 1,1);
        else
            theta2 = angle1;
        end
        tolerance = 1e-6; % 收敛精度
        max_iterations = 100; % 最大迭代次数
        iteration = 0;

        % 牛顿法迭代（防止角度小于盘入点）
        k = 1;
        while true
            theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
            while abs(f(theta2)) > tolerance && iteration < max_iterations
                theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
                iteration = iteration + 1;
            end
            if theta2 > angle1
                break
            else
                theta2 = 1.1^k * angle1;
                k = k + 1;
            end
        end
        end

        % 计算之后点的坐标
        r2 = b * theta2;

```

```

        x2 = r2 * cos(theta2);
        y2 = r2 * sin(theta2);
        x1 = x2;
        y1 = y2;
        theta1 = theta2;
        result(i,1) = theta1;
        result(i,2) = x1;
        result(i,3) = y1;
        result(i,4) = 1;
        hyt2 = 1;
    end
end
end

%% 第三段
if L1 + L2 < t && t <= L1 + L2 + L3

    % 计算第一个点的运动
    theta = (t - L1 - L2) / R;
    x1 = R * cos(theta + Theta_two2) + center2(1);
    y1 = R * sin(theta + Theta_two2) + center2(2);
    result(1,1) = theta + Theta_two2;
    result(1,2) = x1;
    result(1,3) = y1;
    result(1,4) = 3;

    % 第二个点
    hyt2 = 0;
    hyt3 = 0;
    hyt2_first = 0;
    a1 = real(2 * asin(2.86/2/(2*R)));% 盘入圆弧
    a2 = real(2 * asin(2.86/2/R));% 盘出圆弧

    % 预先判断是否在盘入圆弧上
    f = @(theta) sqrt((center1(1) + 2 * R * cos(theta) - x1)^2 + (center1(2) - 2 * R * sin(theta) - y1)^2) - 2.86;
    f_prime = @(theta) (f(theta + 1e-6) - f(theta)) / 1e-6;

    % 初始猜测值
    theta = Theta_one1 + (Theta_one2 - Theta_one1)/2;
    tolerance = 1e-6; % 收敛精度 22
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

```

```
% 牛顿法迭代
```

```
while abs(f(theta)) > tolerance && iteration < max_iterations
```

```
    theta = theta - 0.02 * f(theta) / f_prime(theta);
```

```
    iteration = iteration + 1;
```

```
end
```

```
% 首先判断是否在盘出圆弧上
```

```
if result(1,1) - a2 > Theta_two2 && 2*R > 2.86
```

```
    x2 = R * cos(result(1,1) - a2) + center2(1);
```

```
    y2 = R * sin(result(1,1) - a2) + center2(2);
```

```
    x1 = x2;
```

```
    y1 = y2;
```

```
    result(2,1) = result(1,1) - a2;
```

```
    result(2,2) = x1;
```

```
    result(2,3) = y1;
```

```
    result(2,4) = 3;
```

```
% 再判断是否在盘入圆弧上
```

```
elseif theta > Theta_one1 && theta < Theta_one2 && 4*R > 2.86
```

```
    hyt3 = 1;
```

```
    x2 = 2*R * cos(theta) + center1(1);
```

```
    y2 = - 2*R * sin(theta) + center1(2);
```

```
    x1 = x2;
```

```
    y1 = y2;
```

```
    result(2,1) = theta;
```

```
    result(2,2) = x1;
```

```
    result(2,3) = y1;
```

```
    result(2,4) = 2;
```

```
else
```

```
% 点就在盘入螺旋线上
```

```
% 定义目标函数 f 和数值导数 f_prime
```

```
    hyt2 = 1;
```

```
    hyt3 = 1;
```

```
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2) - y1)^2) - 2.86;
```

```
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数
```

```
% 初始猜测值
```

```
    theta2 = angle1;
```

```
    tolerance = 1e-6; % 收敛精度
```

```
    max_iterations = 100; % 最大迭代次数
```

```
    iteration = 0;
```

```

% 牛顿法迭代（防止角度小于盘入点）
k = 1;
while true
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end
    if theta2 > angle1
        break
    else
        theta2 = 1.1^k * angle1;
        k = k + 1;
    end
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(2,1) = theta1;
result(2,2) = x1;
result(2,3) = y1;
result(2,4) = 1;
end

% 第三个及之后的点
for i = 3:224
    a1 = real(2 * asin(1.65/2/(2*R)));
    a2 = real(2 * asin(1.65/2/R));

    if hyt2 ~= 1
        % 预先判断是否在盘入圆弧上
        f = @(theta) sqrt((center1(1) + 2 * R * cos(theta) - x1)^2 + (center1(2) - 2 * R * sin(theta) - y1)^2) - 1.65;
        f_prime = @(theta) (f(theta + 1e-6) - f(theta)) / 1e-6;

        % 初始猜测值
        theta = Theta_one1 + (Theta_one2 - Theta_one1)/2;
        tolerance = 1e-6; % 收敛精度
        max_iterations = 200; % 最大迭代次数
    end
end

```



```

iteration = 0;

% 牛顿法迭代
while abs(f(theta)) > tolerance && iteration < max_iterations
    theta = theta - 0.02 * f(theta) / f_prime(theta);
    iteration = iteration + 1;
end
if abs(f(theta)) > 0.1
    theta = 100;
end
end

% 首先判断是否在盘出圆弧上
if hyt3 ~= 1 && result(i - 1,1) - a2 > Theta_two2 && 2*R > 1.65
    x2 = R * cos(result(i - 1,1) - a2) + center2(1);
    y2 = R * sin(result(i - 1,1) - a2) + center2(2);
    x1 = x2;
    y1 = y2;
    result(i,1) = result(i - 1,1) - a2;
    result(i,2) = x1;
    result(i,3) = y1;
    result(i,4) = 3;
end

% 再判断是否第一次到盘入圆弧上
elseif hyt2_first ~= 1 && hyt2 ~= 1 && hyt3 ~= 1 && theta > Theta_one1
&& theta < Theta_one2
    hyt3 = 1;
    x2 = 2*R * cos(theta) + center1(1);
    y2 = - 2*R * sin(theta) + center1(2);
    x1 = x2;
    y1 = y2;
    result(i,1) = theta;
    result(i,2) = x1;
    result(i,3) = y1;
    result(i,4) = 2;
    hyt2_first = 1;
end

% 再判断是否在盘入圆弧上
elseif hyt2_first == 1 && hyt2 ~= 1 && result(i - 1,1) - a1 > Theta_one1
&& 4*R > 1.65
    hyt3 = 1;
    x2 = 2*R * cos(result(i - 1,1) - a1) + center1(1);
    y2 = - 2*R * sin(result(i - 1,1) - a1) + center1(2);
    x1 = x2;

```

```

y1 = y2;
result(i,1) = result(i - 1,1) - a1;
result(i,2) = x1;
result(i,3) = y1;
result(i,4) = 2;

else
    % 点就在盘入螺旋线上
    % 定义目标函数 f 和数值导数 f_prime

    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 *
sin(theta2) - y1)^2) - 1.65;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

    % 初始猜测值
    % 上次一是在圆弧上还是螺旋线上
    if hyt2 ~= 0 && hyt3 ~= 0
        theta2 = result(i - 1,1);
    else
        theta2 = angle1;
    end
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代（防止角度小于盘入点）
    k = 1;
    while true
        theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
        while abs(f(theta2)) > tolerance && iteration < max_iterations
            theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
            iteration = iteration + 1;
        end
        if theta2 > angle1
            break
        else
            theta2 = 1.1^k * angle1;
            k = k + 1;
        end
    end

    % 计算之后点的坐标
    r2 = b * theta2;
    x2 = r2 * cos(theta2);

```

```

        y2 = r2 * sin(theta2);
        x1 = x2;
        y1 = y2;
        theta1 = theta2;
        result(i,1) = theta1;
        result(i,2) = x1;
        result(i,3) = y1;
        result(i,4) = 1;
        hyt2 = 1;
        hyt3 = 1;
    end
end
end

%% 第四段
if L1 + L2 + L3 <= t

    % 计算第一个点的运动（盘出螺线）
    S = t - (L1 + L2 + L3);

    theta1 = angle2;
    % 定义目标函数 f 和数值导数 f_prime
    f = @(theta2) (-b/(2*pi)) * (0.5 * theta1 * sqrt(theta1^2 + 1) + 0.5 * log(theta1 + sqrt(theta1^2 + 1)) - 0.5 * theta2 * sqrt(theta2^2 + 1) - 0.5 * log(theta2 + sqrt(theta2^2 + 1))) - S;
    f_prime = @(theta2) - (-b/(2*pi)) * (theta2 / sqrt(theta2^2 + 1) + sqrt(theta2^2 + 1));

    % 初始猜测值
    theta2 = theta1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 300; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end
    r1 = -b * theta2;
    x1 = r1 * cos(theta2);
    y1 = r1 * sin(theta2);
    result(1,1) = theta2;

```

```

result(1,2) = x1;
result(1,3) = y1;
result(1,4) = 4;

% 第二个点
hyt2 = 0;
hyt3 = 0;
hyt4 = 0;
hyt2_first = 0;
hyt3_first = 0;
a1 = real(2 * asin(2.86/2/(2*R)));% 盘入圆弧
a2 = real(2 * asin(2.86/2/R));% 盘出圆弧

% 预先判断是否在盘出圆弧上
f = @(theta_out) sqrt((center2(1) + R * cos(theta_out) - x1)^2 + (center2(2) - R *
sin(theta_out) - y1)^2) - 2.86;
f_prime = @(theta_out) (f(theta_out + 1e-6) - f(theta_out)) / 1e-6;

% 初始猜测值
theta_out = Theta_two2 + (Theta_two1 - Theta_two2)/2;
tolerance = 1e-6; % 收敛精度
max_iterations = 200; % 最大迭代次数
iteration_out = 0;

% 牛顿法迭代
while abs(f(theta_out)) > tolerance && iteration_out < max_iterations
    theta_out = theta_out - 0.02 * f(theta_out) / f_prime(theta_out);% 更新 theta
    iteration_out = iteration_out + 1;
end
if abs(f(theta_out)) > 0.1
    theta_out = 100;
end

% 预先判断是否在盘入圆弧上
f = @(theta_in) sqrt((center1(1) + 2 * R * cos(theta_in) - x1)^2 + (center1(2) - 2 *
R * sin(theta_in) - y1)^2) - 2.86;
f_prime = @(theta_in) (f(theta_in + 1e-6) - f(theta_in)) / 1e-6;

% 初始猜测值
theta_in = Theta_one1 + (Theta_one2 - Theta_one1)/2;
tolerance = 1e-6; % 收敛精度
max_iterations = 200; % 最大迭代次数
iteration_in = 0;

```

```

% 牛顿法迭代
while abs(f(theta_in)) > tolerance && iteration_in < max_iterations
    theta_in = theta_in - 0.02 * f(theta_in) / f_prime(theta_in); % 更新 theta_in
    iteration_in = iteration_in + 1;
end
if abs(f(theta_in)) > 0.1
    theta_in = 100;
end

% 预先判断是否在盘出螺线上
f = @(theta2) sqrt((- b * theta2 * cos(theta2) - x1)^2 + (- b * theta2 * sin(theta2)
- y1)^2) - 2.86;
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

% 初始猜测值
theta2 = result(1,1);
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代
theta2 = theta2 - min(abs(f(theta2) / f_prime(theta2)),1);
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
    iteration = iteration + 1;
end
if theta2 > result(1,1)
    hyt4 = 1;
end

% 首先判断是否在盘出螺线上
if hyt4 ~= 1 && theta2 > angle2
    r2 = -b * theta2;
    x2 = r2 * cos(theta2);
    y2 = r2 * sin(theta2);
    x1 = x2;
    y1 = y2;
    theta1 = theta2;
    result(2,1) = theta1;
    result(2,2) = x1;
    result(2,3) = y1;
    result(2,4) = 4;

% 再判断是否第一次到盘出圆弧上

```

```

elseif hyt3_first ~= 1 && theta_out < Theta_two1 && theta_out > Theta_two2
    hyt4 = 1;
    hyt3_first = 1;
    x2 = R * cos(theta_out) + center2(1);
    y2 = R * sin(theta_out) + center2(2);
    x1 = x2;
    y1 = y2;
    result(2,1) = theta_out;
    result(2,2) = x1;
    result(2,3) = y1;
    result(2,4) = 3;

    % 然后判断是否第一次到盘入圆弧上
elseif hyt2_first ~= 1 && theta_in > Theta_one1 && theta_in < Theta_one2
    hyt3 = 1;
    hyt4 = 1;
    hyt2_first = 1;
    x2 = 2*R * cos(theta_in) + center1(1);
    y2 = - 2*R * sin(theta_in) + center1(2);
    x1 = x2;
    y1 = y2;
    result(2,1) = theta_in;
    result(2,2) = x1;
    result(2,3) = y1;
    result(2,4) = 2;

else
    % 点就在盘入螺线上
    % 定义目标函数 f 和数值导数 f_prime
    hyt2 = 1;
    hyt3 = 1;
    hyt4 = 1;
    f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 * sin(theta2)
    - y1)^2) - 2.86;
    f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;

    % 初始猜测值
    theta2 = angle1;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 100; % 最大迭代次数
    iteration = 0;

    % 牛顿法迭代（防止角度小于盘入点）
    k = 1;

```

```

while true
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end
    if theta2 > angle1
        break
    else
        theta2 = 1.1^k * angle1;
        k = k + 1;
    end
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(2,1) = theta1;
result(2,2) = x1;
result(2,3) = y1;
result(2,4) = 1;
end

% 第三个及之后的点
for i = 3:224
    a1 = real(2 * asin(1.65/2/(2*R)));
    a2 = real(2 * asin(1.65/2/R));

    if hyt4 ~= 1
        % 预先判断是否在盘出螺线上
        f = @(theta2) sqrt((- b * theta2 * cos(theta2) - x1)^2 + (- b * theta2 * sin(theta2) - y1)^2) - 1.65;
        f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6; % 数值导数

        % 初始猜测值
        theta2 = result(i - 1,1);
        tolerance = 1e-6; % 收敛精度
        max_iterations = 100; % 最大迭代次数
        iteration = 0;
    end
end

```

```

% 牛顿法迭代
theta2 = theta2 - min(abs(f(theta2) / f_prime(theta2)),1);
while abs(f(theta2)) > tolerance && iteration < max_iterations
    theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
    iteration = iteration + 1;
end
if theta2 > result(i - 1,1)
    hyt4 = 1;
end
end

if hyt3 ~= 1
    % 预先判断是否在盘出圆弧上
    f = @(theta_out) sqrt((center2(1) + R * cos(theta_out) - x1)^2 +
(center2(2) + R * sin(theta_out) - y1)^2) - 1.65;
    f_prime = @(theta_out) (f(theta_out + 1e-6) - f(theta_out)) / 1e-6;

    % 初始猜测值
    theta_out = Theta_two2 + (Theta_two1 - Theta_two2)/2;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 200; % 最大迭代次数
    iteration_out = 0;

    % 牛顿法迭代
    while abs(f(theta_out)) > tolerance && iteration_out < max_iterations
        theta_out = theta_out - 0.02 * f(theta_out) / f_prime(theta_out); %
更新 theta
        iteration_out = iteration_out + 1;
    end
    if abs(f(theta_out)) > 0.1
        theta_out = 100;
    end
end

if hyt2 ~= 1
    % 预先判断是否在盘入圆弧上
    f = @(theta_in) sqrt((center1(1) + 2 * R * cos(theta_in) - x1)^2 +
(center1(2) - 2 * R * sin(theta_in) - y1)^2) - 1.65;
    f_prime = @(theta_in) (f(theta_in + 1e-6) - f(theta_in)) / 1e-6;

    % 初始猜测值
    theta_in = Theta_one1 + (Theta_one2 - Theta_one1)/2;
    tolerance = 1e-6; % 收敛精度
    max_iterations = 200; % 最大迭代次数

```



```

iteration_in = 0;

% 牛顿法迭代
while abs(f(theta_in)) > tolerance && iteration_in < max_iterations
    theta_in = theta_in - 0.02 * f(theta_in) / f_prime(theta_in);
    iteration_in = iteration_in + 1;
end
if abs(f(theta_in)) > 0.1
    theta_in = 100;
end
end

% 首先判断是否在盘出螺线上
if hyt4 ~= 1 && theta2 > angle2
    r2 = -b * theta2;
    x2 = r2 * cos(theta2);
    y2 = r2 * sin(theta2);
    x1 = x2;
    y1 = y2;
    theta1 = theta2;
    result(i,1) = theta1;
    result(i,2) = x1;
    result(i,3) = y1;
    result(i,4) = 4;

% 再判断是否第一次到盘出圆弧上
elseif hyt3_first ~= 1 && hyt3 ~= 1 && theta_out < Theta_two1 &&
theta_out > Theta_two2
    hyt4 = 1;
    hyt3_first = 1;
    x2 = R * cos(theta_out) + center2(1);
    y2 = R * sin(theta_out) + center2(2);
    x1 = x2;
    y1 = y2;
    result(i,1) = theta_out;
    result(i,2) = x1;
    result(i,3) = y1;
    result(i,4) = 3;

% 再判断是否在盘出圆弧上
elseif hyt3_first == 1 && hyt3 ~= 1 && result(i - 1,1) - a2 > Theta_two2
&& 2*R > 1.65
    hyt4 = 1;
    x2 = R * cos(result(i - 1,1) - a2) + center2(1);

```

```

y2 = R * sin(result(i - 1,1) - a2) + center2(2);
x1 = x2;
y1 = y2;
result(i,1) = result(i - 1,1) - a2;
result(i,2) = x1;
result(i,3) = y1;
result(i,4) = 3;

% 然后判断是否第一次到盘入圆弧上
elseif hyt2_first ~= 1 && hyt2 ~= 1 && theta_in > Theta_one1 && theta_in
< Theta_one2
hyt3 = 1;
hyt4 = 1;
hyt2_first = 1;
x2 = 2*R * cos(theta_in) + center1(1);
y2 = - 2*R * sin(theta_in) + center1(2);
x1 = x2;
y1 = y2;
result(i,1) = theta_in;
result(i,2) = x1;
result(i,3) = y1;
result(i,4) = 2;

% 然后判断是否在盘入圆弧上
elseif hyt2_first == 1 && hyt2 ~= 1 && result(i - 1,1) - a1 > Theta_one1
&& 4*R > 1.65
hyt3 = 1;
hyt4 = 1;
x2 = 2*R * cos(result(i - 1,1) - a1) + center1(1);
y2 = - 2*R * sin(result(i - 1,1) - a1) + center1(2);
x1 = x2;
y1 = y2;
result(i,1) = result(i - 1,1) - a1;
result(i,2) = x1;
result(i,3) = y1;
result(i,4) = 2;

else
% 点就在盘入螺线上
% 定义目标函数 f 和数值导数 f_prime
f = @(theta2) sqrt((b * theta2 * cos(theta2) - x1)^2 + (b * theta2 *
sin(theta2) - y1)^2) - 1.65;
f_prime = @(theta2) (f(theta2 + 1e-6) - f(theta2)) / 1e-6;

```

```

% 初始猜测值
% 上次一是在圆弧上还是螺线上
if hyt2 ~= 0 && hyt3 ~= 0
    theta2 = result(i - 1,1);
else
    theta2 = angle1;
end
tolerance = 1e-6; % 收敛精度
max_iterations = 100; % 最大迭代次数
iteration = 0;

% 牛顿法迭代（防止角度小于盘入点）
k = 1;
while true
    theta2 = theta2 + min(abs(f(theta2) / f_prime(theta2)),1);
    while abs(f(theta2)) > tolerance && iteration < max_iterations
        theta2 = theta2 - 0.02 * f(theta2) / f_prime(theta2);
        iteration = iteration + 1;
    end
    if theta2 > angle1
        break
    else
        theta2 = 1.1^k * angle1;
        k = k + 1;
    end
end

% 计算第二个点的坐标
r2 = b * theta2;
x2 = r2 * cos(theta2);
y2 = r2 * sin(theta2);
x1 = x2;
y1 = y2;
theta1 = theta2;
result(i,1) = theta1;
result(i,2) = x1;
result(i,3) = y1;
result(i,4) = 1;
hyt2 = 1;
hyt3 = 1;
hyt4 = 1;

end
end
end

```

```

overlap = 0;
%% 判断第一个矩形是否与其他矩形重叠
x1 = result(1,2);
y1 = result(1,3);
x2 = result(1 + 1,2);
y2 = result(1 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices1 = [[x1,y1] - 0.15 * N - 0.275 * L;
             [x1,y1] + 0.15 * N - 0.275 * L;
             [x2,y2] + 0.15 * N + 0.275 * L;
             [x2,y2] - 0.15 * N + 0.275 * L;];
for i = 3:49
    x1 = result(i,2);
    y1 = result(i,3);
    x2 = result(i + 1,2);
    y2 = result(i + 1,3);
    L = [x2 - x1, y2 - y1];
    N = [-L(2), L(1)] ./ norm(L);
    L = L ./ norm(L);
    vertices = [[x1,y1] - 0.15 * N - 0.275 * L;
               [x1,y1] + 0.15 * N - 0.275 * L;
               [x2,y2] + 0.15 * N + 0.275 * L;
               [x2,y2] - 0.15 * N + 0.275 * L;];
    if isOverlap(vertices1,vertices) == 1
        overlap = 1;
        break
    end
end
end

```

```

%% 判断第二个矩形是否与其他矩形重叠
x1 = result(2,2);
y1 = result(2,3);
x2 = result(2 + 1,2);
y2 = result(2 + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices1 = [[x1,y1] - 0.15 * N - 0.275 * L;
             [x1,y1] + 0.15 * N - 0.275 * L;
             [x2,y2] + 0.15 * N + 0.275 * L;
             [x2,y2] - 0.15 * N + 0.275 * L;];
for i = 4:49

```

```
x1 = result(i,2);
y1 = result(i,3);
x2 = result(i + 1,2);
y2 = result(i + 1,3);
L = [x2 - x1, y2 - y1];
N = [-L(2), L(1)] ./ norm(L);
L = L ./ norm(L);
vertices = [[x1,y1] - 0.15 * N - 0.275 * L;
            [x1,y1] + 0.15 * N - 0.275 * L;
            [x2,y2] + 0.15 * N + 0.275 * L;
            [x2,y2] - 0.15 * N + 0.275 * L;];
if isOverlap(vertices1,vertices) == 1
    overlap = 1;
    break
end
end
L = L2 + L3;
```
